

AKADEMIKA

DCL-CONV

**CONVOLUTION ENCODER AND
VITERBI DECODER KIT**

EXPERIMENTAL MANUAL

....A MESSAGE FROM



Today's system designers are faced with tomorrow's problems. DIGITAL COMMUNICATION is one of the important subject need to teach while learning electronics.

It is our vision to provide you with the product you need for training ensuring lasting reliability & quality.

OUR MOTTO:

- Light years ahead – refers to leadership.

As leaders in our industry in India, we are totally committed to servicing as the standard against which all are measured in the areas of

• Design • Quality • Value • Delivery • Support.

We are truly light years ahead of our competition in this area. That means that you our valued customers are guaranteed satisfaction.

As you will move through this manual you will quickly discover that we have complete, truly innovative & superior training products. We are so committed to quality that we back our products with a complete comprehensive warranty.

AKADEMIKA LAB SOLUTIONS

Sr. No. 15/8/1, Unit No.9, Kruti Industrial Estate, Opp.

Karishma Society ,Off Karve Road, Kothrud, Pune,

Maharashtra- 411038

INDIA

Contact No. +91-7447438443

Email: lab@akademika.in

www.akademika.in

SAFETY RULES

Carefully follow the instructions contained in this handbook as they give important points on safety during the installation, use and maintenance. Keep this handbook always with you for any further help.

After the unpacking, set all accessories in order so that they are not lost and check the kit integrity. In particular, check that the kit is integral and shows no visible damage.

Before connecting the power supply to the kit, be sure that the jumpers and patch chords are connected correctly as per experiment.

In DCL-CONV kit, signals are to be monitored with an oscilloscope or with help of indicators as explained in the manual.

This kit must be employed only for the use for which it has been conceived, i.e. as educational equipment and must be used under the direct survey of expert personnel. Any other use is improper and so dangerous. The manufacturer cannot be considered responsible for eventual damages due to improper, wrong or unreasonable uses.

In case of any fault/bad operation, turn off the power supply and do not tamper the kit. In case servicing is required, contact the service center for technical assistance.

The kits are liable to malfunction/under-perform if they are not operated under standard environmental conditions of temperature and humidity.

WARRANTY

This kit is warranted against defects in workmanship and materials. Any failure due to defect in either workmanship or material should occur under normal use within a year from the original date of purchase, such failure will be corrected free of charge to the purchaser by repair or replacement of defective part or parts. When the failure is result of user's neglect, natural disaster or accident, we charge for repairs regardless of the warranty period. The warranty does not cover perishable items like connecting chords, Crystals, etc. and other imported items.

This warranty is subject to the following conditions and limitations. The warranty is void and inapplicable if the defective product is not brought or sent to our authorized service center or sales outlet within the warranty period. Defective product is, on Akademika Lab Solutions sole judgment. The defective product will be replaced with new one or repaired without charge or with charge.

In the warranty period if the service is needed, purchaser should get in touch with the service center or sales outlet. The purchaser should return the product to the service center or sales outlet at his or her sole expense. The loss and damage in transit will be outside the preview of this warranty. A returned product must be accompanied by a written description of the defects. Type and Model No. of kits has to be mentioned specifically. We return the product to the purchaser at our expense. In case that warranty does not cover the product on Akademika Lab Solutions judgment, we repair the product after obtaining prior permission from purchaser who receives an estimate statement about repairing charges. In such case, Akademika Lab Solutions bares the transporting expenses required to send back all the repaired products for the moment, and then repairs and transporting expenses will be charged against the purchaser by the statement of accounts.

When the authorized sales agents sell our products, they must notify the purchaser of the warranty contents, but have no rights to stretch the meaning of original warranty contents or offer additional warranty. Akademika Lab Solutions does not provide any other promise or suggestive warranty and hold no liability for the damage caused by negligence, abnormal use or natural disaster. Akademika Lab Solutions is not responsible for the damages even if it is notified of above dangers in advance as well.

For more special service or overall repairs, maintenance and up gradation of products, be sure to contact our service center or sales outlet.

INDEX

Sr. No.	Chapter	Page No.
1	Technical Specifications	5
2	Functional Block	6
3	Introduction	7
4	Circuit Diagram	19
5	Experiment No.1 Study of Convolution Encoding And Hard Decision Viterbi Decoding For K= 7 and Rate =1/2.	25
6	Experiment No.2 Study of Convolution Encoding Using Puncturing For K = 7 and Rate = 3/4.	45
7	Experiment No.3 Study of Convolution Encoding Using Puncturing and Soft Decision Viterbi Decoding Using Depuncturing For K= 7 And Rate = 3/4.	55

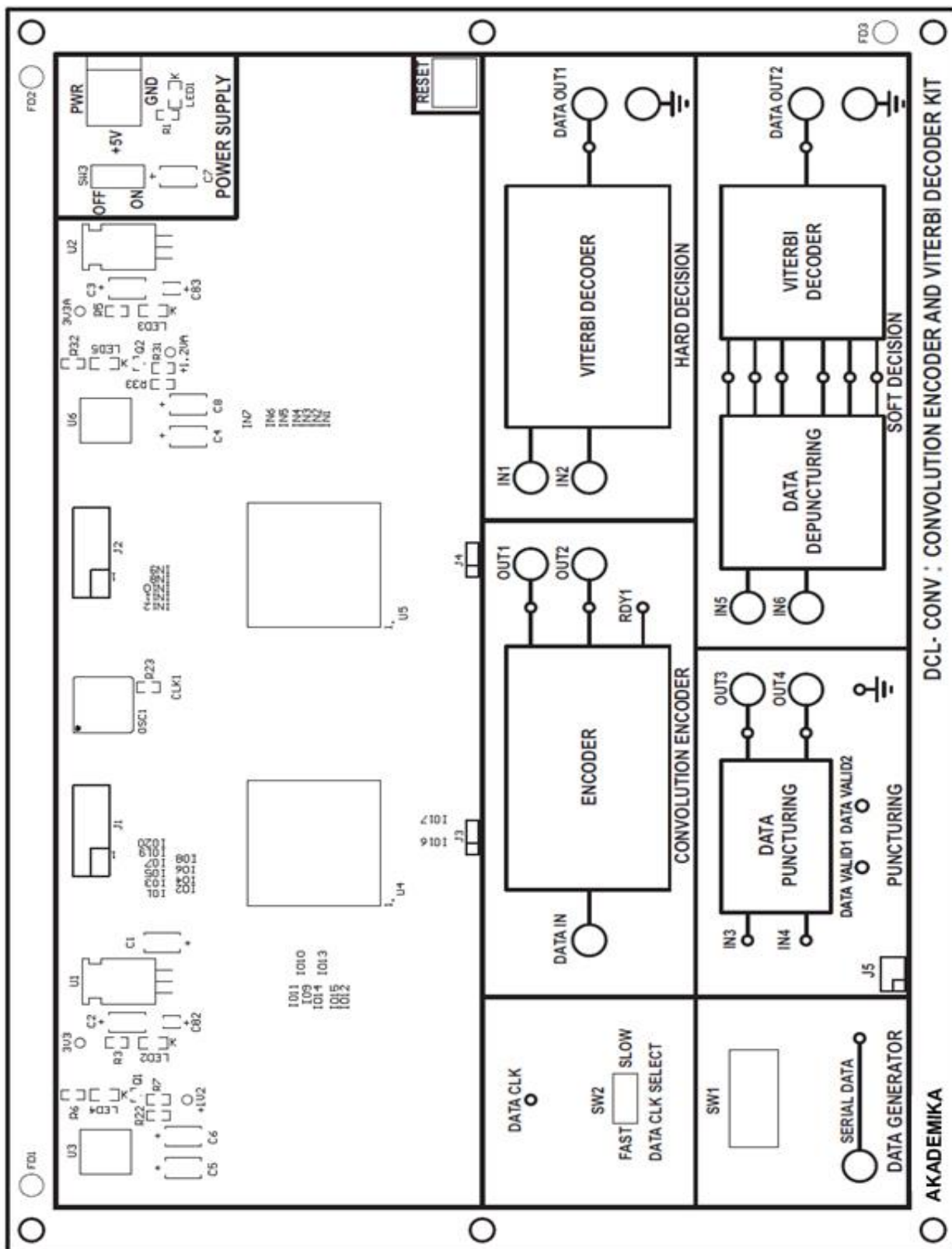
TECHNICAL SPECIFICATIONS

DCL-CONV CONVOLUTION ENCODER AND VITERBI DECODER KIT

DATA SIMULATOR	: Onboard 8-bit variable NRZ-L pattern.
CRYSTAL OSCILLATOR	: 32.768 MHz
DATA CLOCK	: Selectable Data Clock of 256 KHz or 1Hz.
CONSTRAINT LENGTH (K)	: K=7(Convolution Encoder), K=7(Viterbi Decoder).
RATE	: $\frac{1}{2}$ and $\frac{3}{4}$ for both Convolution Encoder and Viterbi Decoder.
MODE OF OPERATION	: Continuous Mode
STANDARD SUPPORTED	: INTELSAT IESS 308/309.
VITERBI DECODER	: Both Hard Decision Viterbi Decoder and Soft Decision Viterbi Decoder. [Soft Decision Viterbi Decoder uses 3-bit soft-quantized demodulated bits]
INTER CONNECTION	: 2 mm banana socket.
POWER SUPPLY	: +7.5V, GND.
TEST POINTS	: 24
ACCESSORIES	

DCL-CONV			Quantity
1	Short Links PCS2 Patch cord	Male To Male	5
2	5V Adapter		1
3	Jumper Caps Small		3
4	Product Manual		1

FUNCTIONAL BLOCK



INTRODUCTION

AKADEMIKA with the trainer, named, “DCL-CONV initiates the user to Convolution Encoding and Viterbi Decoding technique adopted in practice.

“DCL-CONV”: CONVOLUTION ENCODER AND VITERBI DECODER KIT.

FEATURES:

- 1) The trainer comes with an onboard data simulator, which generates the NRZ-L pattern depending on positions of 8-bit switch and the selectable data clock of 256 KHz or 1 Hz which enables the trainers to work in a stand-alone mode. The selectable data clock enables the user to observe the convolution encoding and viterbi decoding both at slow frequency as well as high frequency.
- 2) Convolution encoder for error correction:
 - Selectable rate:
 $K = 7$, rates $\frac{1}{2}$, $\frac{3}{4}$.
 - Continuous mode operation
 - Support for the Intelsat IESS-308/309 standard
 - On-board clock selection
- 3) Viterbi decoder for error correction:
 - Selectable rate :
 $K = 7$, rates $\frac{1}{2}$, $\frac{3}{4}$.
 - Support for Intelsat IESS-308/309 standard
 - Supports both Hard decision and soft decision decoding
 - 3-bit soft-quantized demodulated bits
 - Single 5V supply
- 5) Interconnection facilities:
Sockets and patch chords are provided for on board connections.
- 7) Test Points:
All relevant test points are brought-out for observations. Observations are are viewed on DSO.
- 8) Power Supply:
+7.5V, GND

All communications channels are subject to errors introduced as a result of additive Gaussian noise in their environment. Data perturbations cannot be eliminated but their effect can be minimized by the use of Forward Error Correction (FEC) techniques in the transmitted data stream and decoders in the receiving system that detect and correct bits in error. Improvements in bit error rate performance can be realized for power limited and bandwidth- limited systems typical of satellite communication channels. Using a rate $\frac{1}{2}$ convolutionally encoded systems, approximately 5dB of coding gain can be realized.

Convolutional codes have been in use since the early days of the 1950's. While the implementation of convolutional encoding systems is relatively simple, the process of decoding the resultant data stream at the receiving node can be quite complex. Dr. A. J. Viterbi first described the classic decoder, currently in use, in the late 1960's. This "maximum likelihood" technique greatly improved upon the earlier highly complex methods. Convolutional encoding of data combined with Viterbi decoding at the receiving node is an accepted industry standard for digital channels. Although the Viterbi decoder greatly simplified the process of reconstructing the original data stream, its operation is too complex a subject to be dealt with.

Convolutional codes are frequently used to correct errors in noisy channels. They have rather good correcting capability and perform well even on very bad channels (with error probabilities of about 10^{-3}).

Convolutional codes are extensively used in digital video, radio, mobile communication and satellite communications.

Although convolutional encoding is a simple procedure, decoding of a convolutional code is much more complex task. Several classes of algorithms exist for this purpose:

1. Threshold decoding is the simplest of them, but it can be successfully applied only to the specific classes of convolutional codes. It is also far from optimal.
2. Sequential decoding is a class of algorithms performing much better than threshold algorithms. Their serious advantage is that decoding complexity is virtually independent from the length of the particular code. Although sequential algorithms are also suboptimal, they are successfully used with very long codes, where no other algorithm can be acceptable. The main drawback of sequential decoding is unpredictable decoding latency.
3. Viterbi decoding is an optimal (in a maximum-likelihood sense) algorithm for decoding of a convolutional code. Its main drawback is that the decoding complexity grows exponentially with the code length. So, it can be utilized only for relatively short codes. There are also soft-output algorithms, like SOVA(Soft Output Viterbi Algorithm) or MAP algorithm, which provides not only a decision, but also an estimate of its reliability.

Convolutional Codes

Like any error-correcting code, a convolutional code works by adding some structured redundant information to the user's data and then correcting errors using this information.

Convolutional codes can be systematic and non-systematic. Systematic codes are those where an unencoded sequence is a part of the output sequence. Systematic codes are

almost always recursive; conversely, non-recursive codes are almost always non-systematic. Refer fig (a) and fig (b).

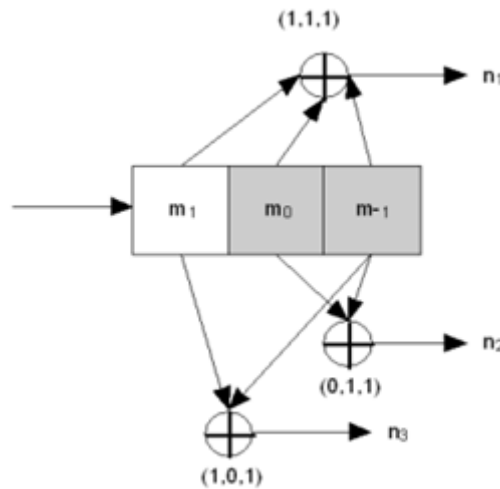


Fig (a) : Rate 1/3 non-recursive, non-systematic convolutional encoder with constraint length 3

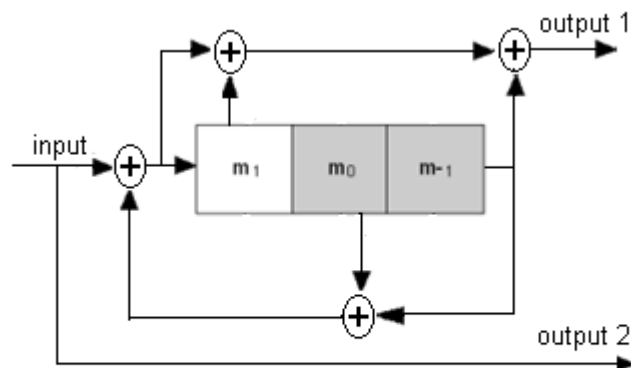


Fig (b) :Rate 1/2 recursive, systematic convolutional encoder

Convolutional Encoders:

A convolutional encoder is a linear system. A binary convolutional encoder can be represented as a shift register. The outputs of the encoder are modulo 2 sums of the values in the certain register's cells. The input to the encoder is either the unencoded sequence (for non-recursive codes) or the unencoded sequence added with the values of some register's cells (for recursive codes).

The drawing below depicts the convolutional encoder typically used in conjunction with an $R=1/2$, $K=7$ Viterbi Decoder. The intent of this note is to help clarify the terms used to define the convolutional encoding.

Constraint Length = K

The term “K” defines the length or number of stages (n) in the shift register. In the drawing below there are seven stages to the shift register; therefore, K=7.

Rate (R) = m/n

The “R” term speaks of the output rate (frequency) relative to the input. In an R=1/2 system, the output rate (n) is twice the input rate (m). Stated in another way, two bits are output for every single bit that is input. In an R=2/3 system, three bits are output for every two bits that are input.

Gn = ()

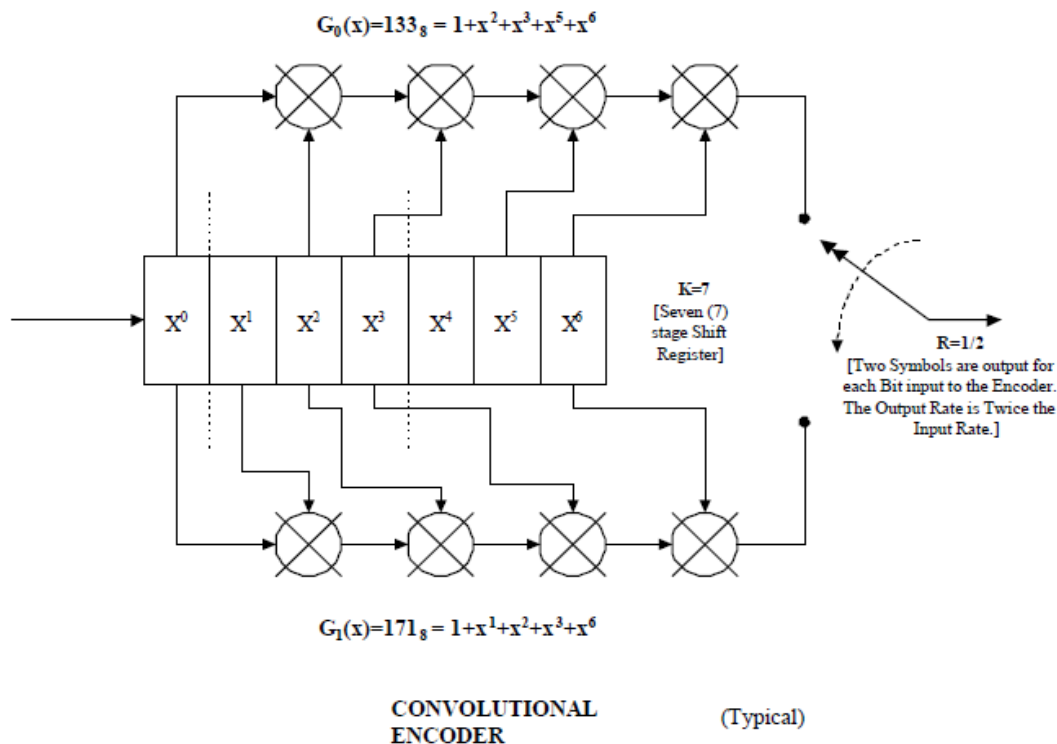
The terms G0 and G1 (sometimes referred to as G1 and G2) represent a shorthand that defines the location of the taps on the shift register. As shown in the drawing, G0 = 133 represents the octal code for the upper connections to the shift register while G1 = 171 describes the lower connections.

For example, for the decoder on the Figure 1, the polynomials are:

- $G_0(x) = 1+x^2+x^3+x^5+x^6$
- $G_1(x) = 1+x+x^2+x^3+x^6$

Encoder polynomials are usually denoted in the octal notation. For the above example, these designations are “1011011” = 133₈ and “1111001” = 171₈.

The constraint length of this code is 7.



(Figure 1)

Puncturing:

After convolutional encoding, some of the encoded symbols can be selectively removed before transmission. This process, called “puncturing,” is a data compression method used to reduce the number of bits transmitted.

Puncturing is a technique used to make m/n rate code from a “basic” rate code. It is reached by deletion of some bits in the encoder output. Bits are deleted according to puncturing matrix. A “1” in the puncturing matrix means the corresponding symbol is kept in the output stream, while a “0” means the symbol in that position is removed.

Punctured convolutional codes:

Punctured convolutional codes are derived from ordinary codes by dropping some output bits (i.e. not sending them to a channel). The resulting code has higher rate (less redundancy) but lower correcting capability. The order of dropping is defined by a puncturing matrix. The number of rows in a puncturing matrix is equal to the number of output polynomials. The elements of this matrix are 1's (which means that we transmit a given bit) and 0's (which means that we drop it). The puncturing matrix is applied to the output stream cyclically. The standard puncturing matrices for the rate $\frac{1}{2}$, $K = 7$ (INTELSAT) convolutional codes are:

Code rate	Generator polynomial	Puncturing Matrix
Rate 2/3	$G_0 = 133_8$ $G_1 = 171_8$	11 10
Rate 3/4	$G_0 = 133_8$ $G_1 = 171_8$	110 101
Rate 5/6	$G_0 = 133_8$ $G_1 = 171_8$	11010 10101
Rate 7/8	$G_0 = 133_8$ $G_1 = 171_8$	1111010 1000101

Fig (c): $R = \frac{1}{2}$, $K=7$ (Intelsat) puncturing matrix

For example, consider a puncturing matrix for the rate 2/3. It means that we will transmit only every second output bit from the first polynomial, but every bit from the second polynomial. Fig (d) given below shows an example of puncturing process.

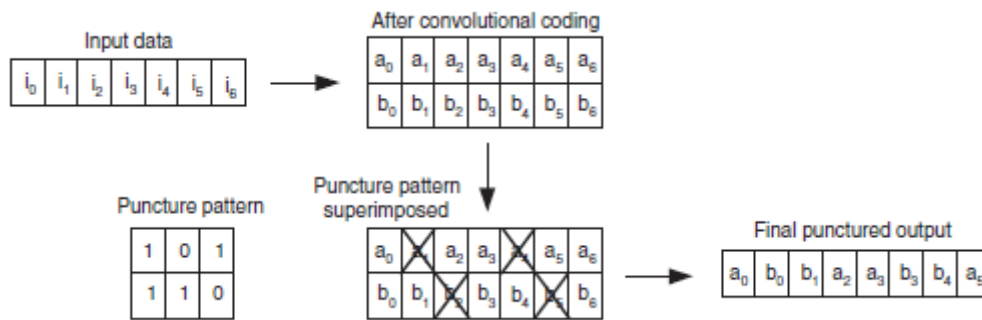


Fig (d): Puncturing process

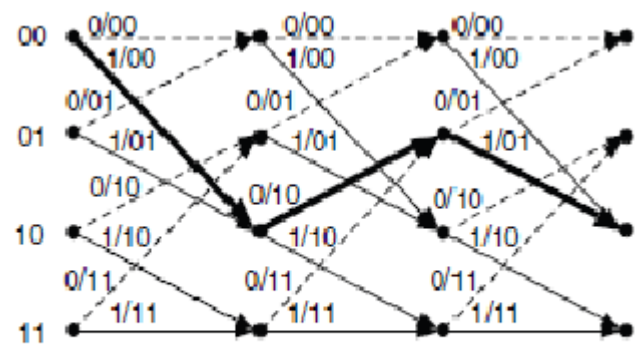
Depuncturing:

If puncturing is employed in the encoder, the decoder will have to “depuncture” the data before decoding. Depuncturing is done by inserting NULL symbols for the punctured symbols. NULL symbols are equidistant from both ‘0’ and ‘1’. In order to decode a punctured convolutional code, one needs first to insert erasure marks to the places where punctured symbols should be. This task is performed by a depuncturer. A pair of binary strings, called a “puncture pattern,” is used to identify punctured symbols. A “1” in a pattern means the corresponding symbol was not punctured in the encoder, while a “0” means the symbol has been punctured.

Then, the Viterbi decoder should ignore the erased bits during branch metric calculation.

Viterbi Decoding:

The convolutional encoding mentioned above can be considered as a series of state transitions for every input symbol. The input and the resulting state transitions can be shown in a special state transition diagram called a “trellis tree” or simply a “trellis.” A sample trellis tree is shown in Fig (e).



Trellis for 3 stages and constraint length = 3
 Branches corresponding to input seq. 101 is highlighted

In the above trellis, the branches for three transitions are drawn. The path of the trellis for a typical input sequence, 101, is highlighted in the figure. Any transmission error alters the path traversed in the trellis. In Viterbi decoding, such a trellis is formed in memory, where the metrics corresponding to all paths are recorded. After constructing the trellis for a sufficient length (called the trace back length, L), the trace back process starts from node 0 in the last state. During the trackback process, the original sequence is reconstructed.

from the trellis. In error-prone applications, however, a trellis of length $2L$ is constructed and two trace back processes are employed. The first trace back starts from node 0, traces back L stages of the trellis, and ends up in a node which is more likely to be the right starting point for the second trace back. The second trace back starts from this reliable starting point and traces back another L node. The data corresponding to the second trace back are decoded to result in the original data stream.

Hard versus Soft Decisions:

Before specifying an algorithm that will determine the maximum likelihood decision, let us describe the channel. The code word sequence $U^{(m)}$, made up of branch words with each branch word comprised of n code symbols, can be considered to be an endless stream as opposed to a block code, in which the source data and their code words are partitioned into precise block sizes.

The code word sequence shown in fig (e) emanates from the convolution encoder and enters the modulator, where the code symbols are transformed into signal waveforms. The modulation may be baseband (e.g. pulse waveform) or band pass (e.g. PSK or FSK). In general, if ' L ' symbols at a time, where ' L ' is an integer, are mapped into signal waveforms $s_i(t)$, where $i = 1, 2, \dots, M = 2^L$. When $L = 1$, the modulator maps each code symbol into a binary waveform. The channel over which the waveform is transmitted is assumed to corrupt the signal with Gaussian noise. When the corrupted signal is received it is first processed by the demodulator and then by the decoder.

Consider that a binary signal transmitted over a symbol interval $(0, T)$ is represented by $s_1(t)$ for a binary one and $s_2(t)$ for binary zero. The received signal is $r_i(t) = s_i(t) + n(t)$, where $n(t)$ is a zero mean Gaussian noise process. The detection of $r(t)$ is in terms of two basic steps :

1. The received waveform is reduced to a single number, $z(T) = a_i + n_0$ where a_i is the signal component of $z(T)$ and n_0 is the noise component. The noise component, n_0 , is the zero mean Gaussian Random Variable, and thus $z(T)$ is Gaussian Random Variable with a mean of either a_1 or a_2 depending on whether a binary one or zero was sent.
2. In the second step of the detection process a decision was made as to which signal was transmitted on the basis of comparing $z(T)$ to a threshold.

The conditional probabilities of $z(T)$, $p(z/s_1)$ and $p(z/s_2)$ are shown in fig (f) labeled likelihood of s_1 and likelihood of s_2 . The demodulator in the fig (d) converts the set of time ordered random variable $\{z(T)\}$ into a code sequence ' Z ' and passes it to the decoder. The demodulator output can be configured in variety of ways. It can be implemented to make a hard decision as to whether $z(T)$ represents a zero or one. In this case the output of demodulator is quantized to two levels, zero and one, and fed into the decoder. Since the decoder operates on the hard decisions made by the demodulator, the decoding is called hard decision decoding.

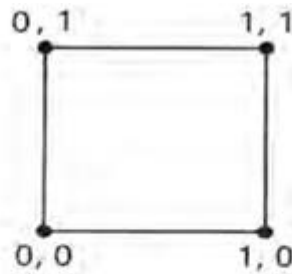


Fig (d) Hard-decision plane

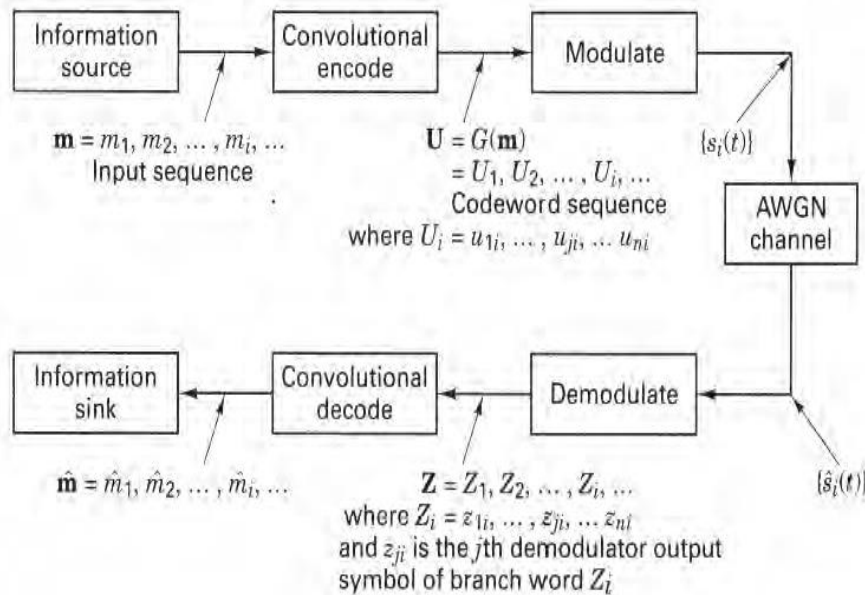


Fig (e): Encode/Decode and modulate/demodulate portions of a communication link
The demodulator can also be configured to feed the decoder with a quantized value of $z(T)$ greater than two levels. Such an implementation furnishes the decoder with more information than is provided in the hard decision case. When the quantization level of input is greater than two, the decoding is called soft decision decoding.

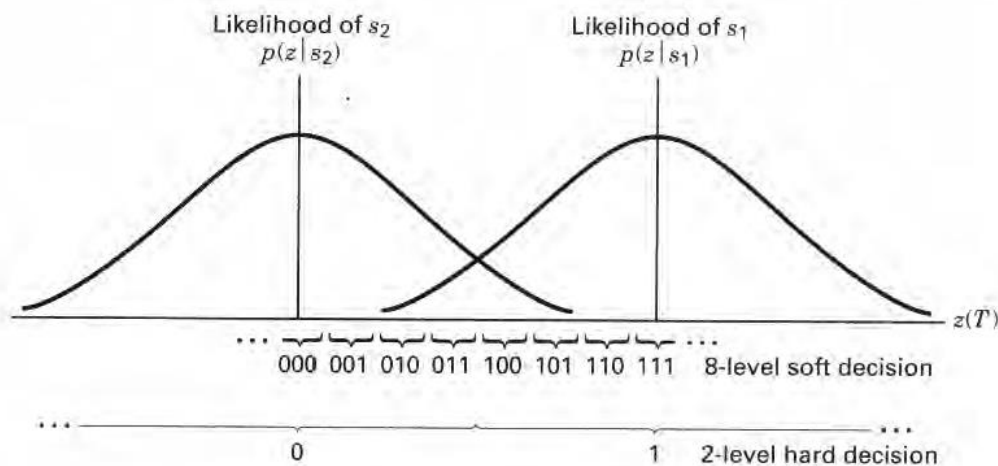


Fig (f): Hard and Soft Decoding Decisions

Eight levels (3-bits) of quantization are illustrated on the abscissa of fig (f). When the demodulator sends a hard binary decision to the decoder, it sends it a single binary symbol. When the demodulator sends a soft binary decision, quantized to eight levels it sends the decoder a 3-bit word describing an interval along $z(t)$. In effect, sending such a 3-bit word in place of a single binary symbol is equivalent to sending the decoder a measure of confidence along with the code symbol decision.

Referring to fig (f) if the demodulator sends 111 to the decoder, this is tantamount to declaring the code symbol to be a one with very high confidence, while sending a 100 is tantamount to declaring the code symbol to be a one with very low confidence. The idea behind the demodulator not making hard decisions and sending more data (soft decisions) to the decoder can be thought of as an interim step to provide the decoder with more information which the decoder then uses for recovering the message sequence. In fig (f), the 8-level soft decision metric is often shown as -7, -5, -3, -1, 1, 3, 5, 7. Such a designation lends itself to a simple interpretation of the soft decision: The sign of the metric represents a decision (e.g. choose s_1 if positive choose s_2 if negative), and the magnitude of the metric represents the confidence level of that decision. The only advantage of the metric shown in fig (d) is that it avoids the use of negative numbers.

Block decoding algorithms and convolutional decoding algorithms has been devised to operate with hard or soft decisions. However, soft decision decoding is generally not used with block codes because it is considerably more difficult than hard decision decoding to implement. The most prevalent use of soft decision decoding is with the Viterbi convolutional decoding algorithm, since with Viterbi decoding, soft decision represents only a trivial increase in computation.

Viterbi Convolutional Decoding Algorithm:

The Viterbi decoding algorithm was discovered and analyzed by Viterbi in 1967. The Viterbi algorithm essentially performs maximum likely decoding; however it reduces the computational load by taking advantage of special structure in the code Trellis. The algorithm involves calculating a measure of similarity, or distance, between the received signal, at time t_i . The Viterbi algorithm removes from consideration those trellis paths that could not possibly be candidates for maximum likelihood choice. When two paths enter the same state, the one having the best metric is chosen; this path is called the surviving path. This selection of surviving path is performed for all the states. The decoder continues in this way to advance deeper into the trellis, making decisions by eliminating the least likely paths. The early rejection of the unlikely path reduces the decoding complexity. In 1969 Omura demonstrated that the viterbi algorithm is, in fact, maximum likelihood. Note that the goal of selecting the optimum path can be expressed, equivalently, as choosing the codeword with the minimum distance metric.

Hard decision Viterbi decoding:

Hard decision viterbi decoding is hamming distance decoding. It can be implemented using Trellis diagram. According to viterbi algorithm maximum one incoming path can be selected for each state. The selected path is called survivor path. The Hamming distance between the received bits and path is called metric. The minimum metric path is survivor path. If two paths are available with same minimum metric select any one path randomly. In a received code the last $K-1$ sets are due to padded zeroes. So in Trellis diagram for the

last K-1 stages select only solid lines. From the final survivor path decoded bits can be generated.

Soft decision Viterbi decoding:

For a rate 1/2 convolutional code system the demodulator delivers two code symbols at a time to the decoder. For hard-decision (2-level) decoding, each pair of received code symbol can be depicted on a plane as one of the corners of a square, as shown in fig (1). The corners are labeled with the binary numbers (0,0), (0,1), (1,0), (1,1), representing the four possible hard decision values that the two code symbols might have. For 8-level soft decision decoding, each pair of code symbols can be similarly represented on an equally spaced 8-level by 8-level plane, as a point from the set of 64 points shown in fig (2) given below. In this soft decision case the demodulator no longer delivers firm decisions; it delivers quantized noisy signals (soft decisions).

The primary difference between hard decision and soft decision Viterbi decoding is that soft decision algorithm cannot use a Hamming distance metric because of its limited resolution. A distance metric with the needed resolution Euclidean distance, and to facilitate its use, the binary numbers of 1 and 0 are transformed to octal numbers 7 and 0, respectively. This can be seen in fig (3), where the corners of the square have been re-labeled accordingly this allows us to use a pair of integers, each in the range of 0 to 7, for describing any point in the 64-point set. Also shown in the fig (4) is the point 5, 4, representing an example of a pair of noisy code-symbol values that might stem from a demodulator. Imagine that the square in fig (3) has co-ordinates x and y. Then, what is the Euclidean distance between the noisy point 5,4 and the noiseless point 0, 0? It is $\sqrt{(5-0)^2 + (4-0)^2} = \sqrt{41}$. Similarly, if we ask what is the Euclidean distance between the noisy point 5, 4 and the noiseless point 7, 7? It is $\sqrt{(5-7)^2 + (4-7)^2} = \sqrt{13}$.

Soft-decision Viterbi decoding, for the most part proceeds in the same way as hard-decision decoding. The only difference is that Hamming distances are not used. Consider how soft-decision decoding is performed with the use of Euclidean distances. Fig (4) shows the first section of encoding trellis, originally presented in fig (g) with the branch words transformed from binary to octal. Suppose that a pair of soft decision code symbol with values (5, 4) arrives at a decoder during the first transition interval. Fig (e) shows the first sequence of a decoding trellis. The metric ($\sqrt{41}$), representing the Euclidean distance between the arriving 5, 4 and the 0, 0 branch word, placed on the solid line. Similarly the metric ($\sqrt{13}$), representing the Euclidean distance between the arriving 5, 4 and the 7, 7 code symbols is placed on the dashed line. The rest of the task, pruning the trellis in search of a common stem proceeds in the same way as hard decision decoding.

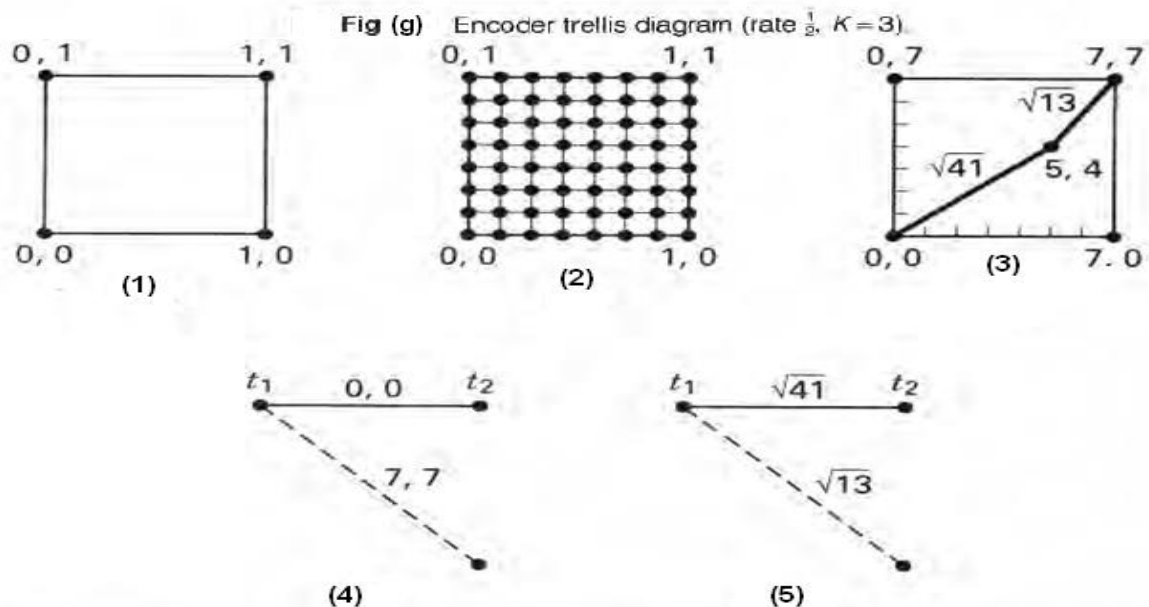
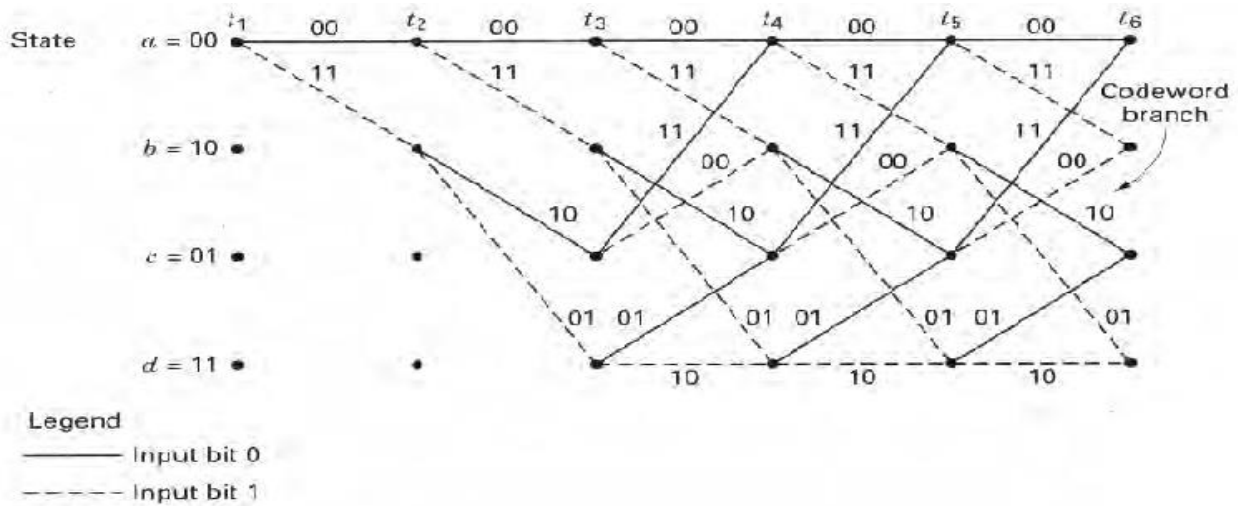


Fig (1) Hard-decision plane (2) 8-level by 8-level soft-decision plane (3) Example of soft code symbols (4) Encoding trellis section (5) Decoding trellis section.

Circuit Description

• Clock And Data Generator

This block generates NRZ-L data of a variable pattern using VLSI technology depending on positions of eight bit switch SW1 also Reference clock of frequency 256KHz or 1 Hz (DATA CLK) is generated. The crystal oscillator generates a 32.768MHz clock, from which a 256 KHz or 1 Hz clock is generated by using down counter. For data generation output of SW1 is given to Parallel to serial shift register. Output of shift register is taken as Reference Data. The data clock of 256 KHz or 1 Hz can be selected by using switch SW2. When the switch SW2 is moved towards fast then data clock of 256 KHz is selected. When the switch SW2 is moved towards slow then data clock of 1 Hz is selected.

- **Convolution Encoder Section**

The logic for the convolution encoder is written in VHDL and the convolution encoded data is out from the FPGA on OUT 1 and OUT 2 pin.

- **Hard Decision Viterbi Decoder Section**

The logic for the Hard Decision Viterbi Decoder is written in VHDL. This section decodes the convolution encoded data, and the decoded data is out from the FPGA on Data Out1 pin.

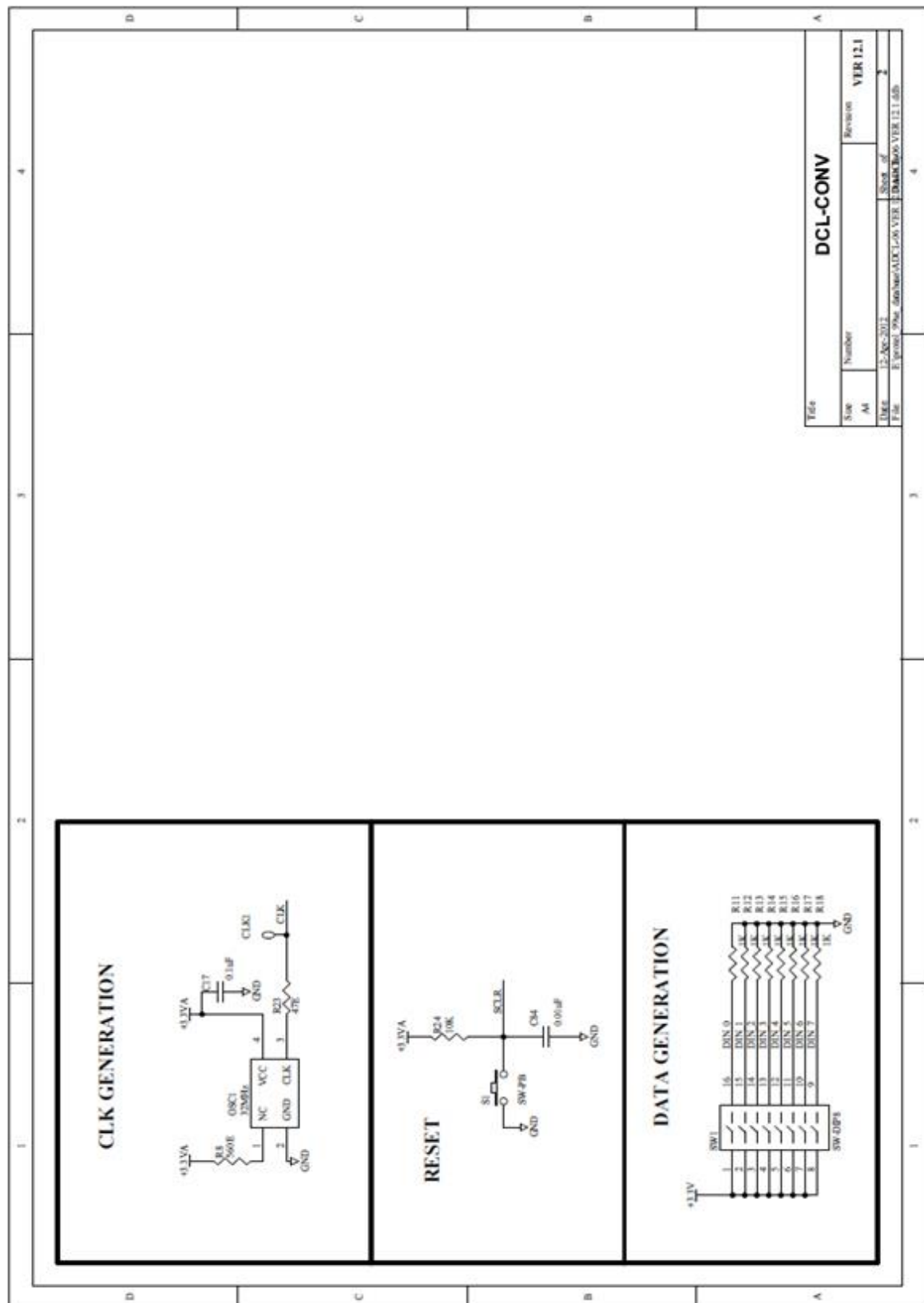
- **Data Puncturing Section**

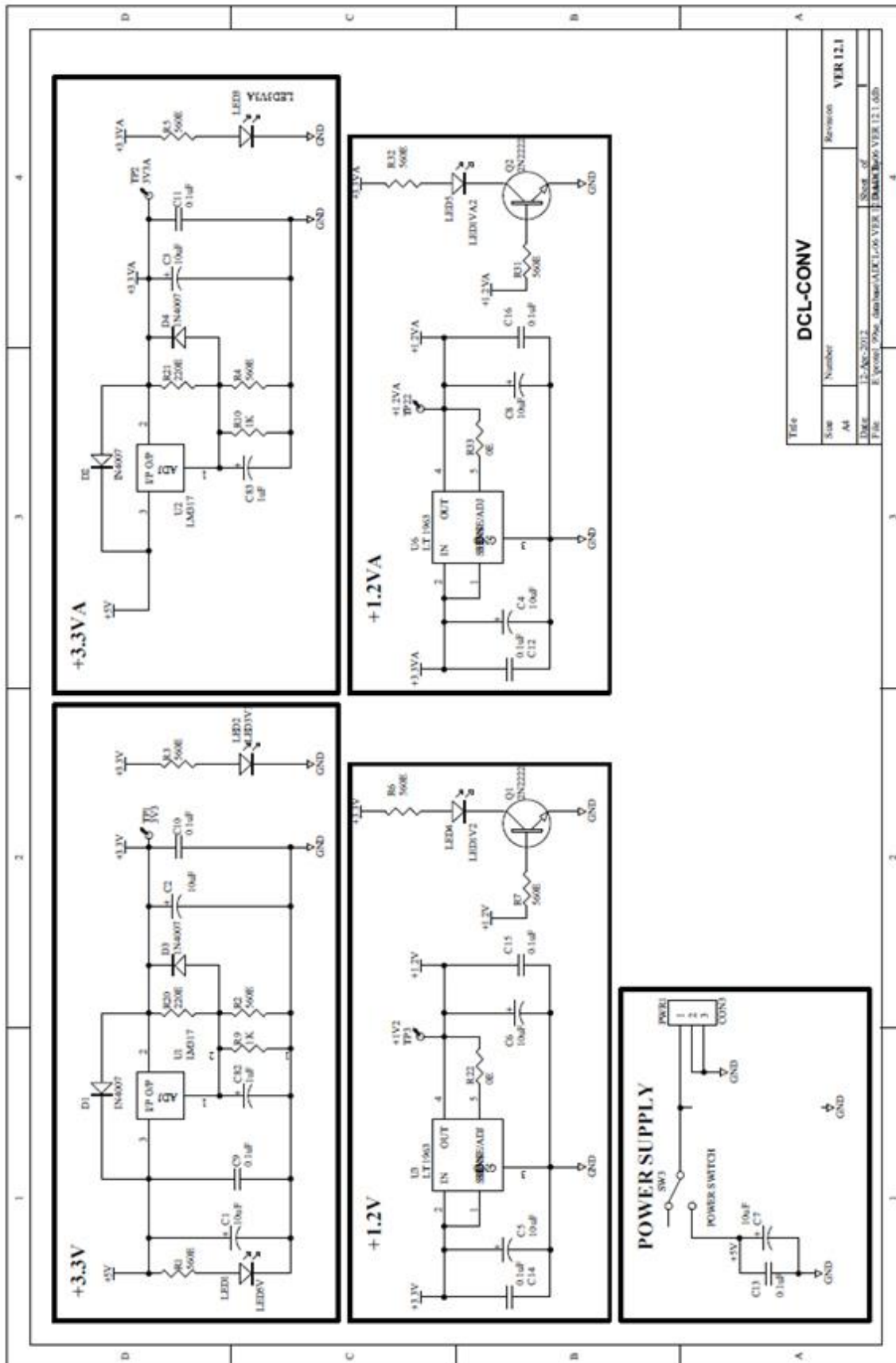
The convolution encoded data is punctured using data puncturing section. The logic for data puncturing is written in VHDL and the punctured convolution encoded data is out from FPGA on OUT 3 and OUT 4 pin.

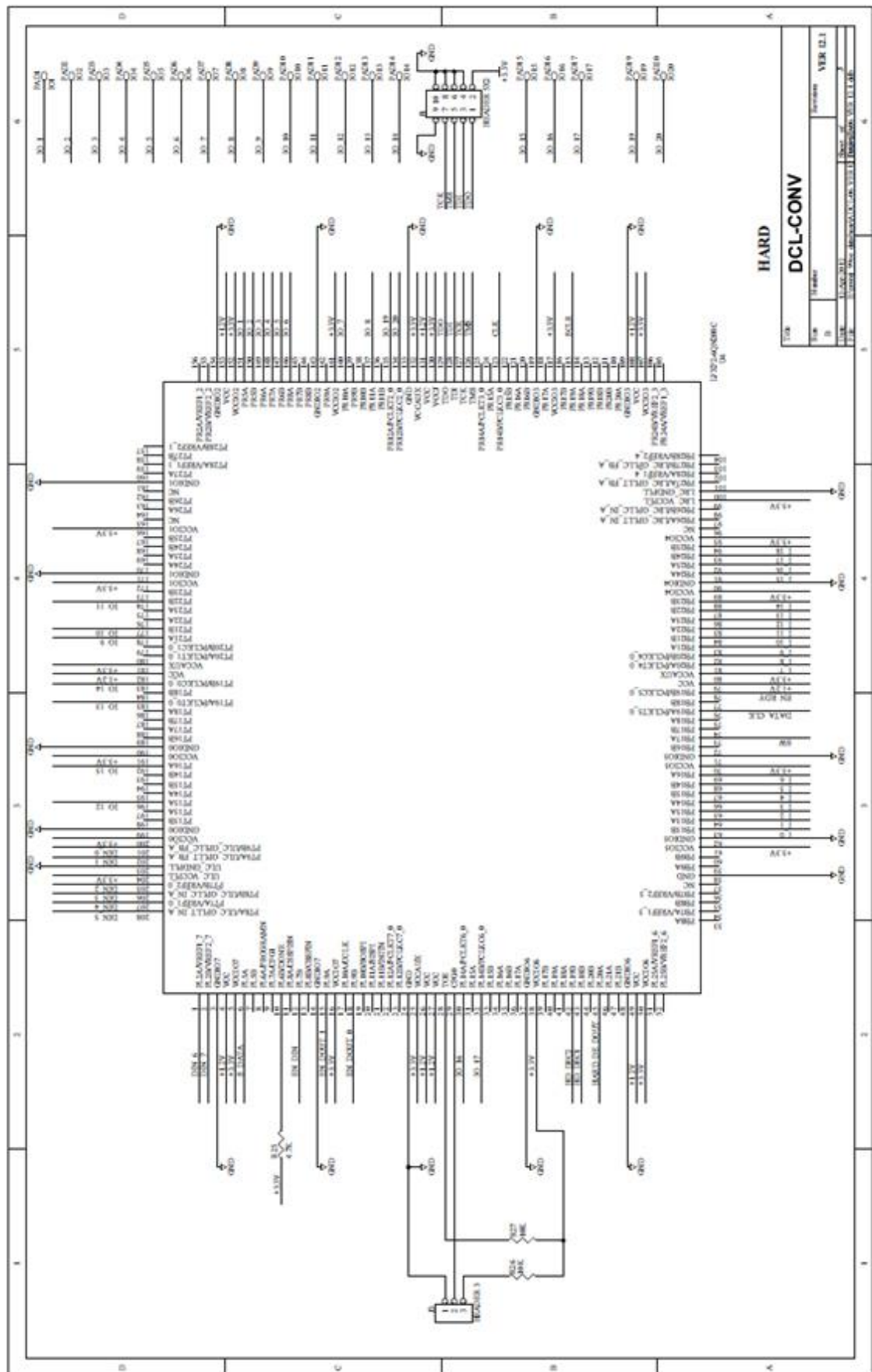
- **Soft Decision Viterbi Decoder Section**

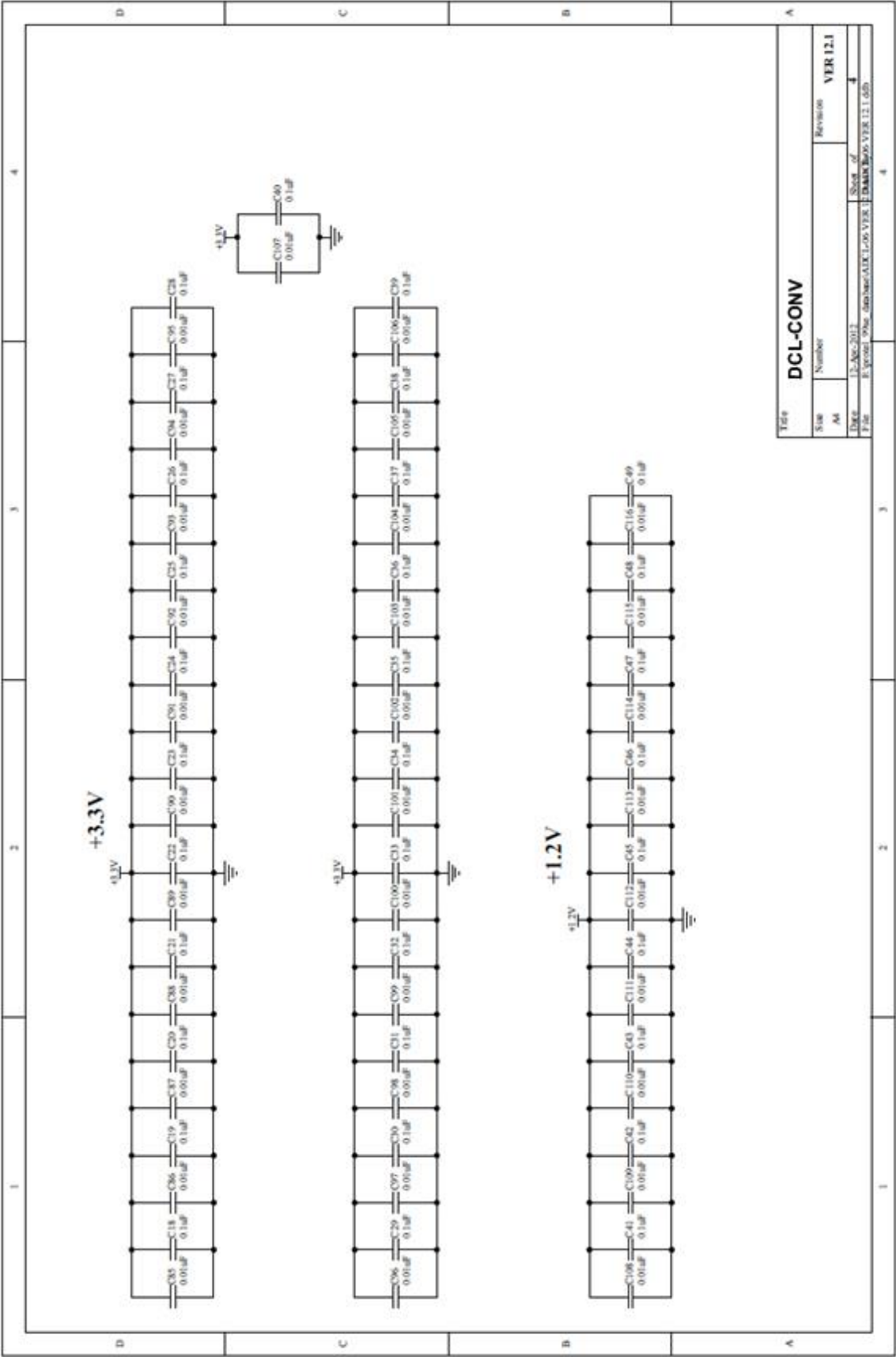
The punctured convolution encoded data is depunctured using data depuncturing block and the depunctured data is decoded using the Viterbi Decoder in Soft Decision Viterbi Decoder section. The logic for this entire section is written in VHDL and the decoded data is out on Data Out 2 pin.

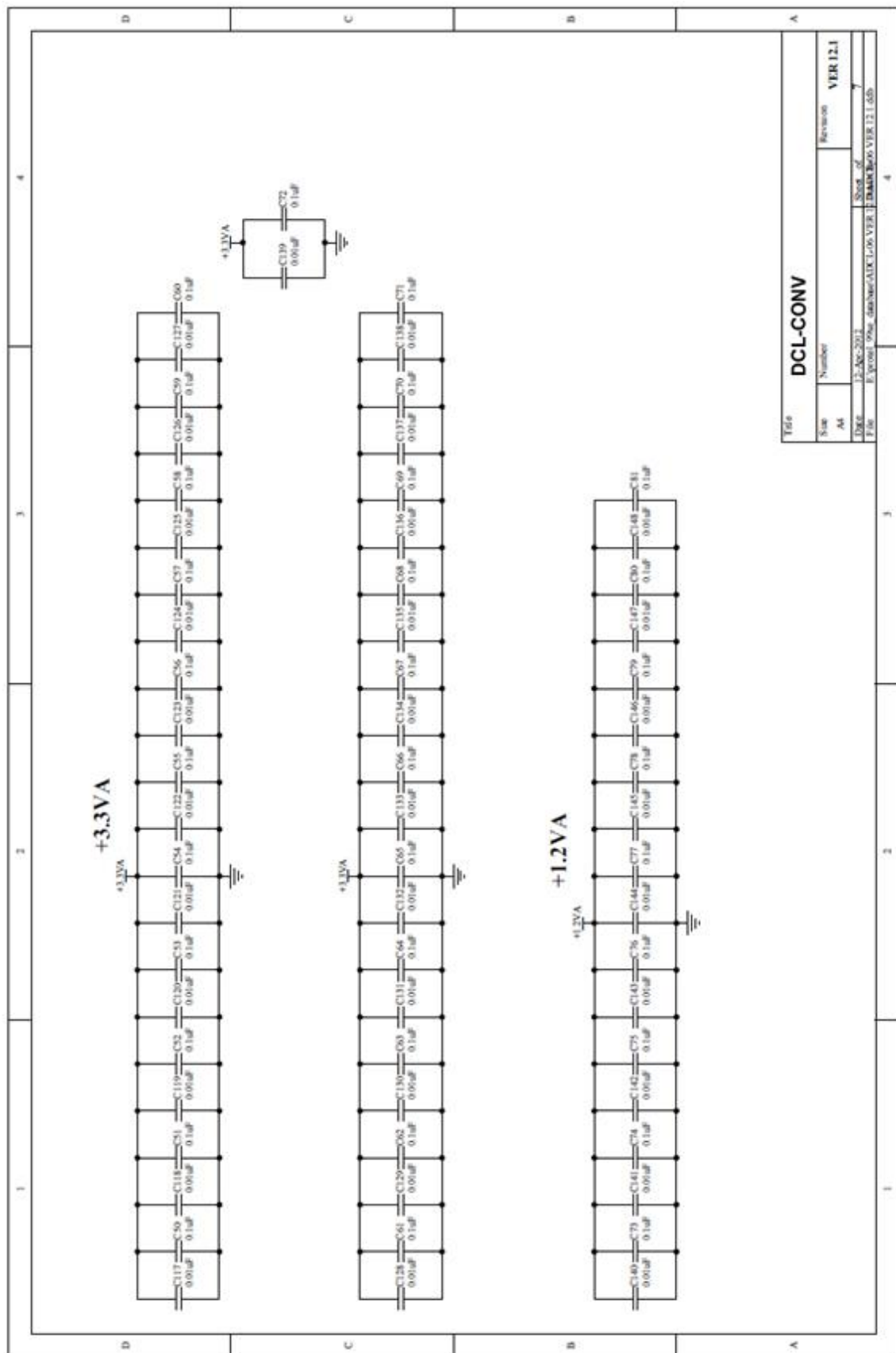
CIRCUIT DIAGRAM OF DCL-CONV

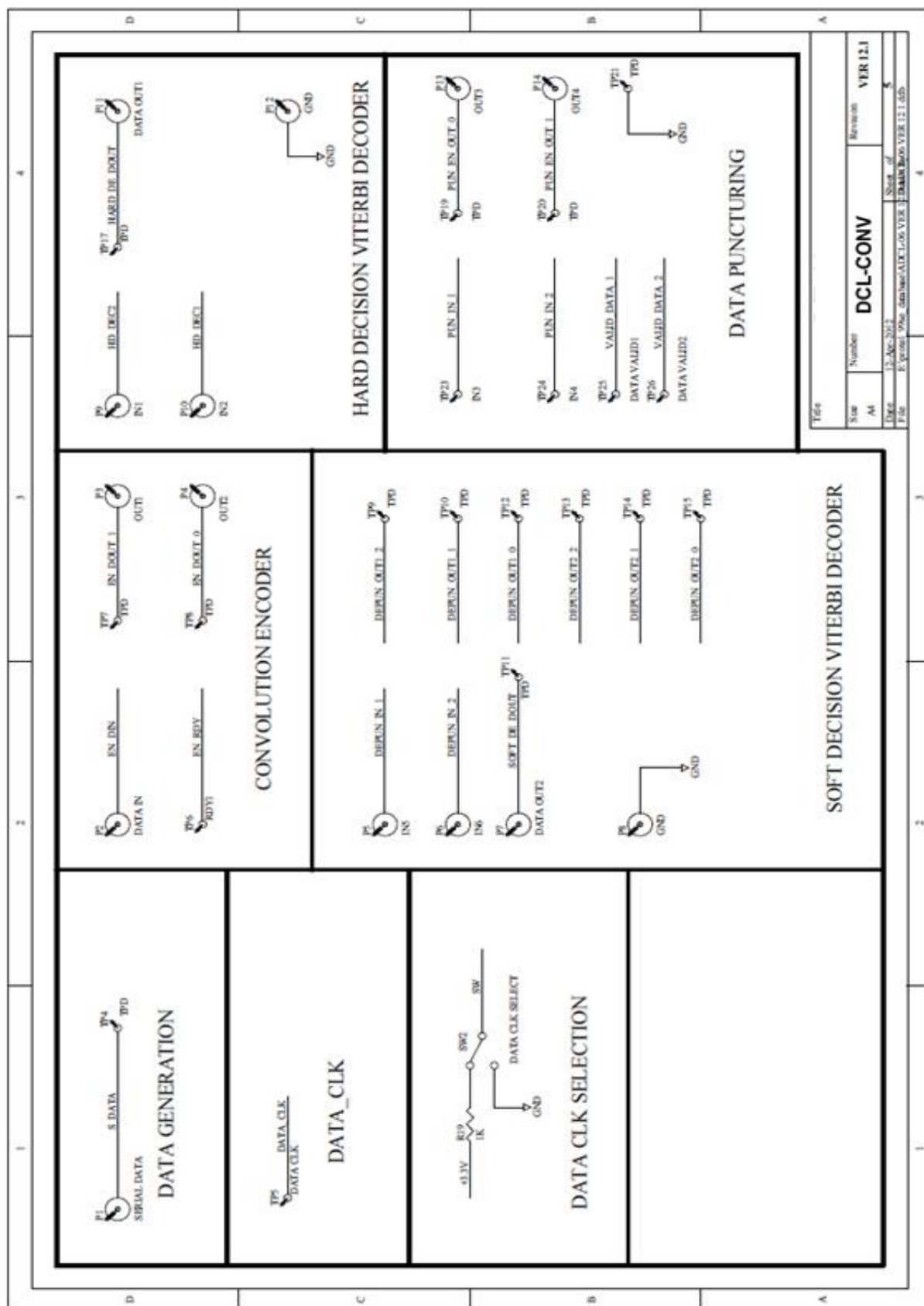












EXPERIMENT NO. 1

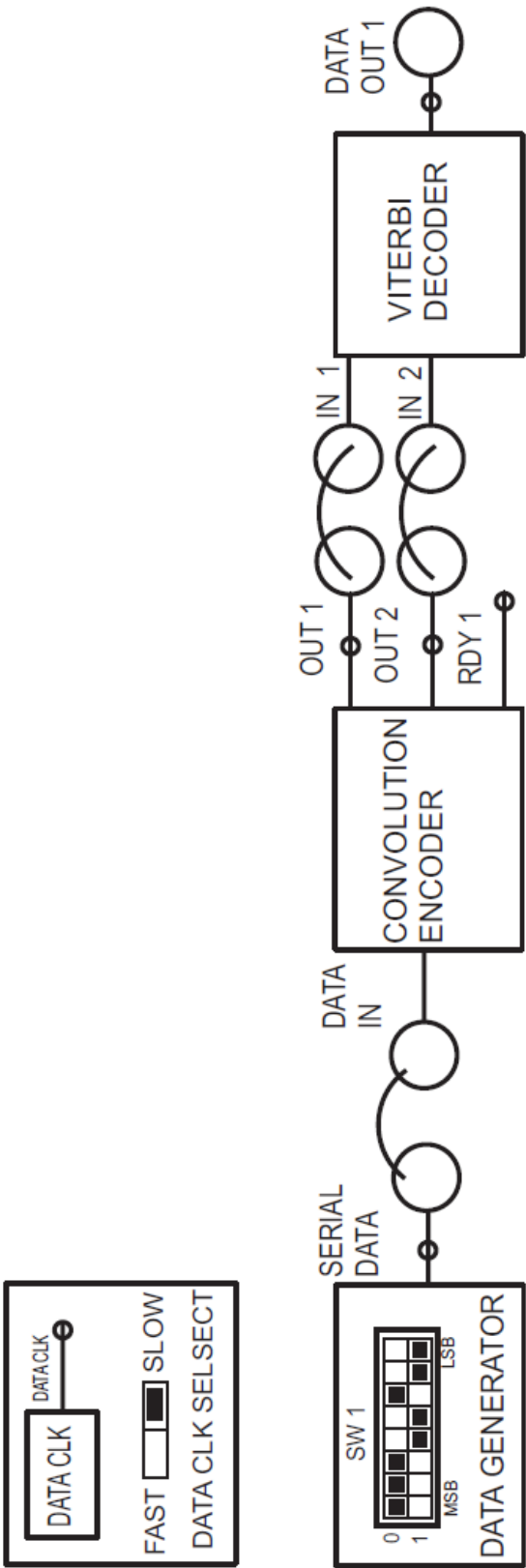


FIG 1.1 BLOCK DIAGRAM FOR
STUDY OF CONVOLUTION ENCODER AND HARD DECISION
VITERBI DECODER FOR K=7, RATE = $\frac{1}{2}$

EXPERIMENT NO: 1

NAME:

Study Of Convolution Encoding And Hard Decision Viterbi Decoding For K= 7 And Rate =1/2.

OBJECTIVE:

To Study Convolution Encoding And Hard Decision Viterbi Decoding For K= 7 And Rate =1/2.

THEORY:

A convolutional code works by adding some structured redundant information to the user's data and then correcting errors using this information. A convolutional encoder is a linear system. A binary convolutional encoder can be represented as a shift register. The outputs of the encoder are modulo 2 sums of the values in the certain register's cells. The input to the encoder is either the unencoded sequence (for non-recursive codes) or the unencoded sequence added with the values of some register's cells (for recursive codes).

The convolution encoder used in DCL-CONV supports INTELSAT standard. As per the standard the generator polynomials for K = 7, R = 1/2 are

$$G_0(x) = 1+x^2+x^3+x^5+x^6$$

$$G_1(x) = 1+x+x^2+x^3+x^6$$

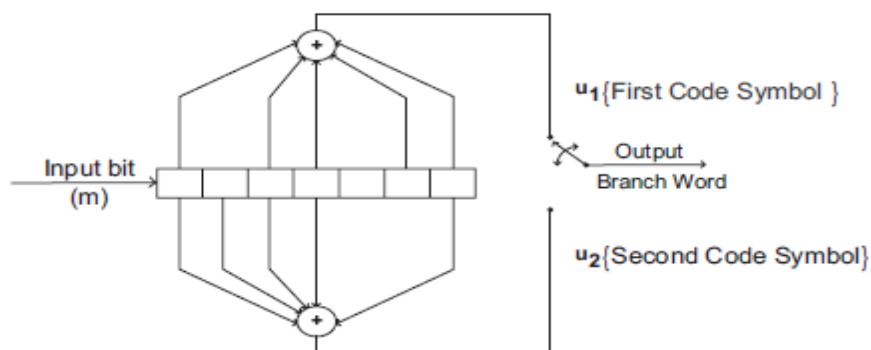


Fig (a)

i.e. $G_0(x) = 133_{(\text{octal})}$ & $G_1(x) = 171_{(\text{octal})}$. The implementation depicted below and is used in conjunction with an R=1/2, K=7 Hard Decision Viterbi Decoder. The intent of this experiment is to help clarify the terms used to define the convolutional encoding and Viterbi decoding as well as to explain how convolutional encoding and Hard decision Viterbi decoding takes place theoretically and to observe and verify the results practically.

We can approach the encoder in terms of its impulse response i.e. the response of the encoder to a single “one” bit that moves through it. Consider the contents of the register in Fig (a)

Register contents	Branch word	
	U1	U2
1 0 0 0 0 0 0	1	1
0 1 0 0 0 0 0	0	1
0 0 1 0 0 0 0	1	1
0 0 0 1 0 0 0	1	1
0 0 0 0 1 0 0	0	0
0 0 0 0 0 1 0	1	0
0 0 0 0 0 0 1	1	1

Fig (b)

Input sequence: 1 0 0 0 0 0 0

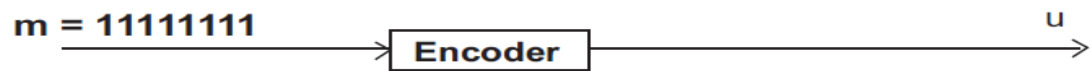
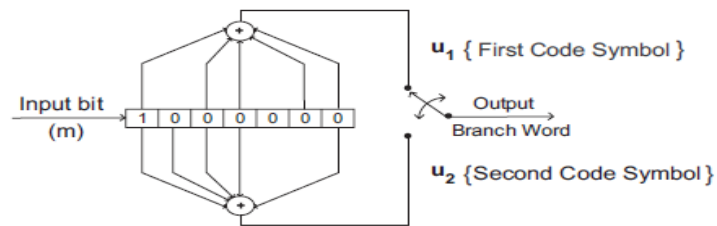
Output sequence: 11 01 11 11 00 10 11

The output sequence for the input “one” is called the impulse response of the encoder. Then for the input sequence $m = 1\ 1\ 1\ 1\ 1\ 1\ 1$, the output may be found by the superposition or the linear addition of the time shifted input “impulses” as follows:

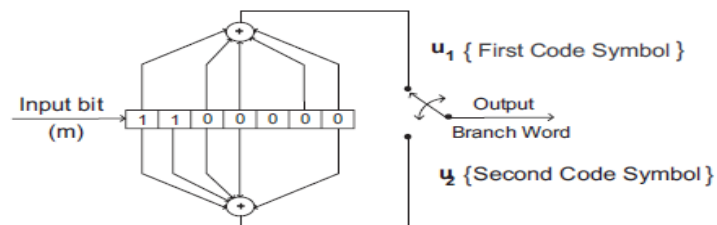
Input (m)	Output													
1	11	01	11	11	00	10	11							
1		11	01	11	11	00	10	11						
1			11	01	11	11	00	10	11					
1				11	01	11	11	00	10	11				
1					11	01	11	11	00	10	11			
1						11	01	11	11	00	10	11		
1							11	01	11	11	00	10	11	
1								11	01	11	11	00	10	11
Modulo-2 sum	11	10	01	10	10	00	11	11	00	01	10	01	01	11

Fig (c)

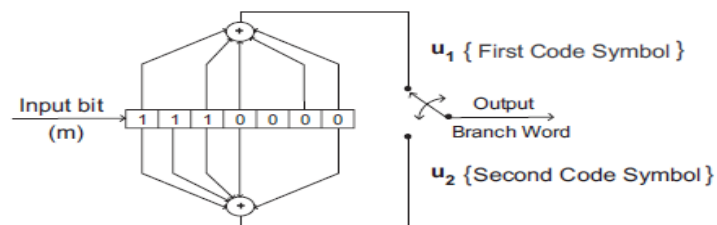
Observe that this is the same output obtained in Fig (d), demonstrating that convolutional codes are linear. It is from this property of generating the output by the linear addition of time shifted impulses, or the convolution of the input sequence with the impulse response of the encoder, that we derive the name convolutional encoder.

**Time****Output** t_1 

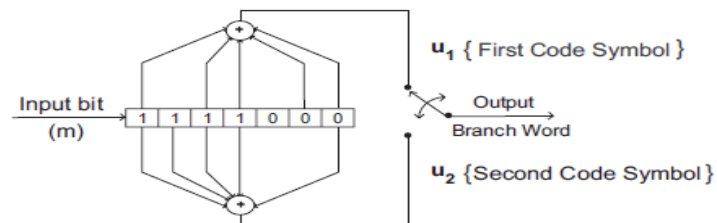
u_1	u_2
1	1

 t_2 

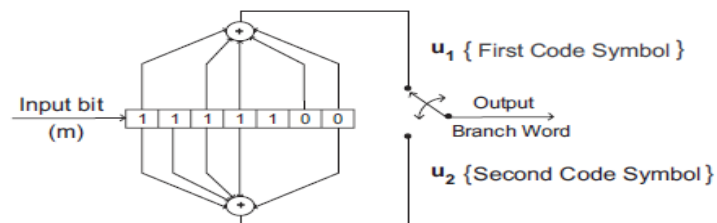
u_1	u_2
1	0

 t_3 

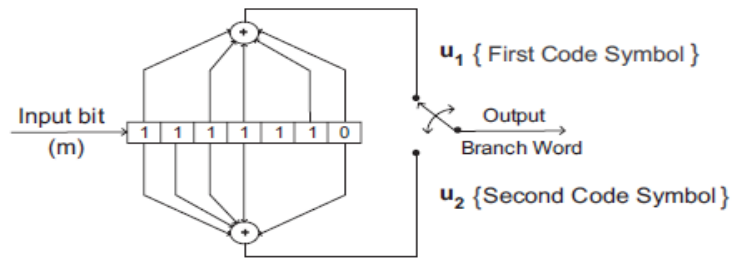
u_1	u_2
0	1

 t_4 

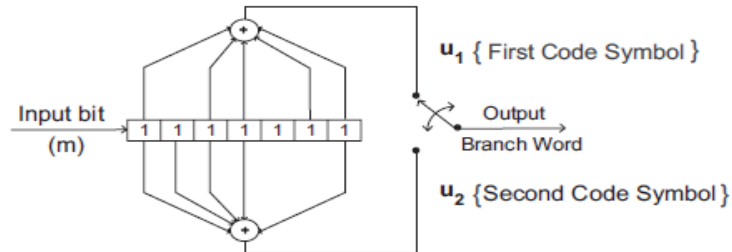
u_1	u_2
1	0

 t_5 

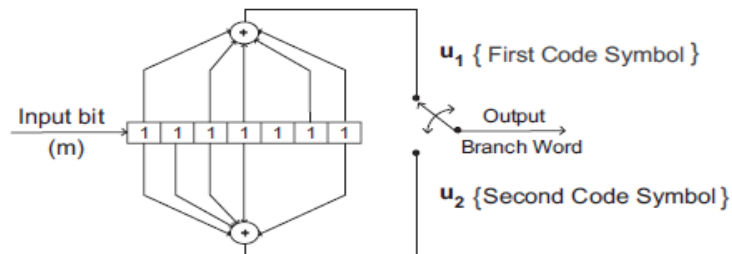
u_1	u_2
1	0

Time**Output****t₆**

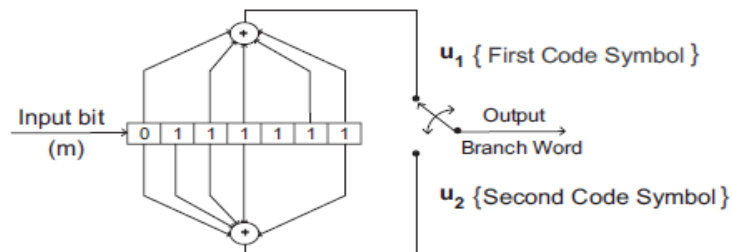
u₁ { First Code Symbol }
0 **u₂**
0

t₇

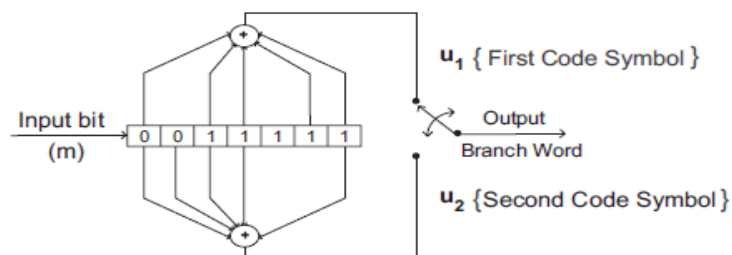
u₁ { First Code Symbol }
1 **u₂**
1

t₈

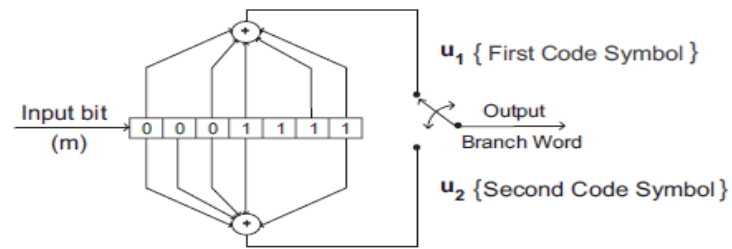
u₁ { First Code Symbol }
1 **u₂**
1

t₉

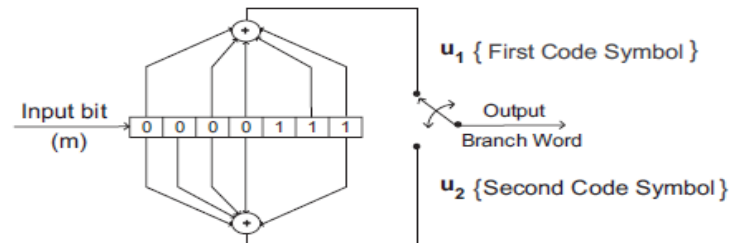
u₁ { First Code Symbol }
0 **u₂**
0

t₁₀

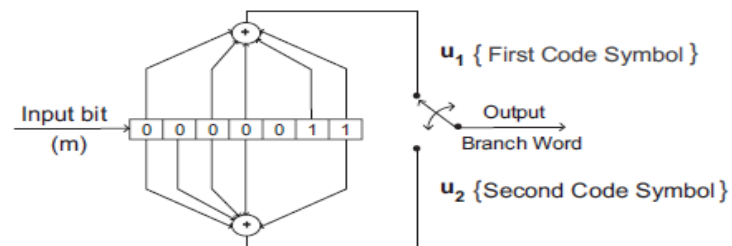
u₁ { First Code Symbol }
0 **u₂**
1

Time**Output** **t_{11}** 

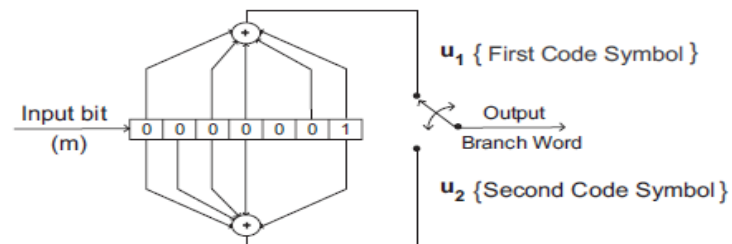
u_1	u_2
1	0

 t_{12} 

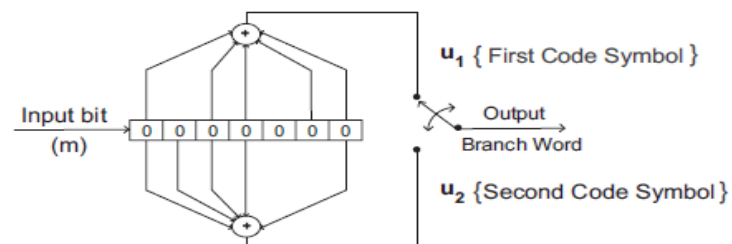
u_1	u_2
0	1

 t_{13} 

u_1	u_2
0	1

 t_{14} 

u_1	u_2
1	1

 t_{15} 

u_1	u_2
0	0

Output Sequence : 11 10 01 10 10 00 11 11 00 01 10 01 01 11

Fig (d)

An example of Convolution Encoding technique:

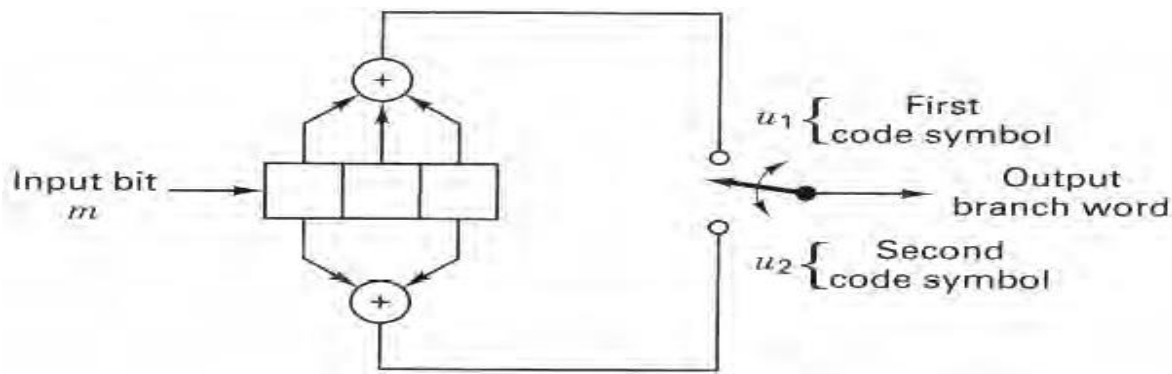


Fig (e): Convolutional Encoder (rate $\frac{1}{2}$, $K = 3$)

The Fig (e) illustrates a (2, 1) convolutional encoder with constraint length $K = 3$. There are $n = 2$ modulo-2 adder; thus the code rate $k/n = \frac{1}{2}$. At each input bit time, a bit is shifted into the leftmost stage and the bits in the register are shifted one position to the right. Next, the output switch samples the output of each modulo-2 adder (i.e. first the upper adder, then the lower adder), thus forming the code symbol pair making up the branch word associated with the bit just inputted. The sampling is repeated with for each inputted bit. The choice of connection between the adders and the stages of the register gives rise to the characteristics of the code. Any change in the choice of connections results in a different code. The connections are, of course, not chosen or changed arbitrarily. The problem of choosing connections having good distance properties is complicated and has not been solved in general; however, good codes have been found with the help of computer search for all constraint length less than 20.

We can approach the encoder in terms of its impulse response i.e. the response of the encoder to a single “one” bit that moves through it. Consider the contents of the register in Fig (e)

Register contents	Branch word	
	U1	U2
1 0 0	1	1
0 1 0	1	0
0 0 1	1	1

Fig (f)

Input sequence: 1 0 0
Output sequence: 11 01 11

The output sequence for the input “one” is called the impulse response of the encoder. Then for the input sequence $m = 1\ 1\ 0\ 1\ 1$, the output may be found by the superposition or the linear addition of the time shifted input “impulses” as follows:

Input (m)	Output						
1	11	10	11				
1		11	10	11			
0			00	00	00		
1				11	10	11	
1					11	10	11
Modulo-2 sum	11	01	01	00	01	01	11

Fig (g)

Hard decision Viterbi's decoding of above example:

Before proceeding towards the hard decision viterbi decoding of above example let us discuss about the state representation and state diagram of Convolutional Encoder.

State Representation and State diagram:

A convolution encoder belongs to a class of devices known as finite-state machines, which the general name is given to machines that have a memory of past signals. The adjective finite refers to the fact that there are only a finite number of unique states that the machine can encounter. In the most general sense, the state consists of the smallest amount of information that, together with a current input to the machine, can predict the output of the machine.

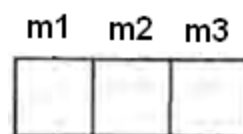
For rate $1/n$ convolutional encoder, the state is represented by, the rightmost $K - 1$ stages.

Since the rightmost $K - 1$ stages of the register in the example of convolution encoder represented by Fig (e) are 2. Hence there are $2^2 = 4$ possible states for the encoder. These states of the encoder are represented as $a = 00$, $b = 10$, $c = 01$, $d = 11$.

State diagram can be drawn from the state transition table. Before proceeding further let us focus on how the state transition table is prepared.

State transition Table:

For simplicity let us designate the registers of the convolution encoder described by Fig (e) as $m1$, $m2$, $m3$.



Since there are 4 states, there arise 4 cases for the input states.

Case 1:

Input state is a. Since a is represented as $a = 00$ then $m_2 = 0$ and register $m_3 = 0$.

m1	m2	m3
	0	0

Now the register m_1 can either be 0 or 1. If $m_1 = 0$ then

register content	output state	output bit				
		U1	U2			
m1 m2 m3						
<table border="1"><tr><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	a	0	0
0	0	0				

Note that the content of the register m_1 and m_2 decide the output state, since $m_1=0$, $m_2=0$ the output state is a.

Similarly if the input state is a and if $m_1 = 1$ then

register content	output state	output bit				
		U1	U2			
m1 m2 m3						
<table border="1"><tr><td>1</td><td>0</td><td>0</td></tr></table>	1	0	0	b	1	1
1	0	0				

Case 2:

Similarly if the input state is b then using the steps mentioned in case 1

register content	output state	output bit							
		U1	U2						
<table><tr><td>m1</td><td>m2</td><td>m3</td></tr><tr><td>0</td><td>1</td><td>0</td></tr></table>	m1	m2	m3	0	1	0	c	1	0
m1	m2	m3							
0	1	0							
<table><tr><td>m1</td><td>m2</td><td>m3</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	m1	m2	m3	1	1	0	d	0	1
m1	m2	m3							
1	1	0							

Case 3:

Similarly if the input state is c then using the steps mentioned in case 1

register content	output state	output bit				
		U1	U2			
m1 m2 m3						
<table border="1"><tr><td>0</td><td>0</td><td>1</td></tr></table>	0	0	1	a	1	1
0	0	1				
m1 m2 m3						
<table border="1"><tr><td>1</td><td>0</td><td>1</td></tr></table>	1	0	1	b	0	0
1	0	1				

Case 4:

Similarly if the input state is d then using the steps mentioned in case 1

register content	output state	output bit				
		U1	U2			
m1 m2 m3						
<table><tr><td>0</td><td>1</td><td>1</td></tr></table>	0	1	1	c	0	1
0	1	1				
m1 m2 m3						
<table><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	d	1	0
1	1	1				

Now using the above cases the state transition table can be prepared as:

Input state	Input bit	Output state	Output bit
a	0	a	00
	1	b	11
b	0	c	10
	1	d	01
c	0	a	11
	1	b	00
d	0	c	01
	1	d	10

Fig (h)

State Diagram:

One way to represent the simple encoder is with a state diagram; such a representation for the encoder in Fig (e) is shown in Fig (i). The states, shown in the boxes of the diagram, represent the possible contents of the rightmost $K - 1$ stages of the register, and the paths between the states represent the output branch words resulting from such state transitions. The state of the register are designated $a = 00$, $b = 10$, $c = 01$, and $d = 11$; the diagram shown in Fig (i) illustrates all the state transitions that are possible for the encoder in Fig (e). There are only two transitions emanating from each state, corresponding to the two possible input bits. Next to each path between states is written the output branch word associated with the state transition. In drawing the path, we use the convention that a solid line denotes a path associated with an input bit, one. Notice that it is not possible in a single transition to move from a given state to any arbitrary state. As a consequence of shifting-in one bit at a time, there are only two possible state transitions that the register can make at each bit time. For example, if the present encoder state is 00, the only possibilities for the state at the next shift are 00 or 10.

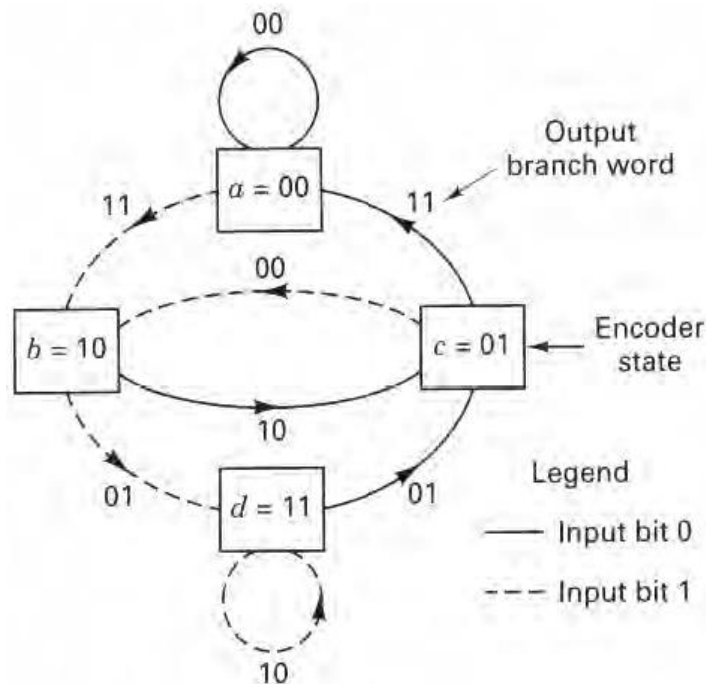


Fig (i): Encoder state diagram (rate = $1/2$, $K = 3$)

Trellis Diagram:

In drawing the trellis diagram, we use the same convention that we introduced with the state diagram – a solid line denotes the output generated by an input bit zero, and a dashed line denotes the output generated by an input bit one. The nodes of the trellis characterize the encoder states; the first row nodes correspond to the state $a = 00$, the second and subsequent rows correspond to the states $b = 10$, $c = 01$ and $d = 11$. At each unit of time, the trellis requires 2^{k-1} nodes to represent the 2^{k-1} possible encoder states. The trellis in our example assumes a fixed periodic structure after trellis depth 3 is reached (at time t_4). In the general case,

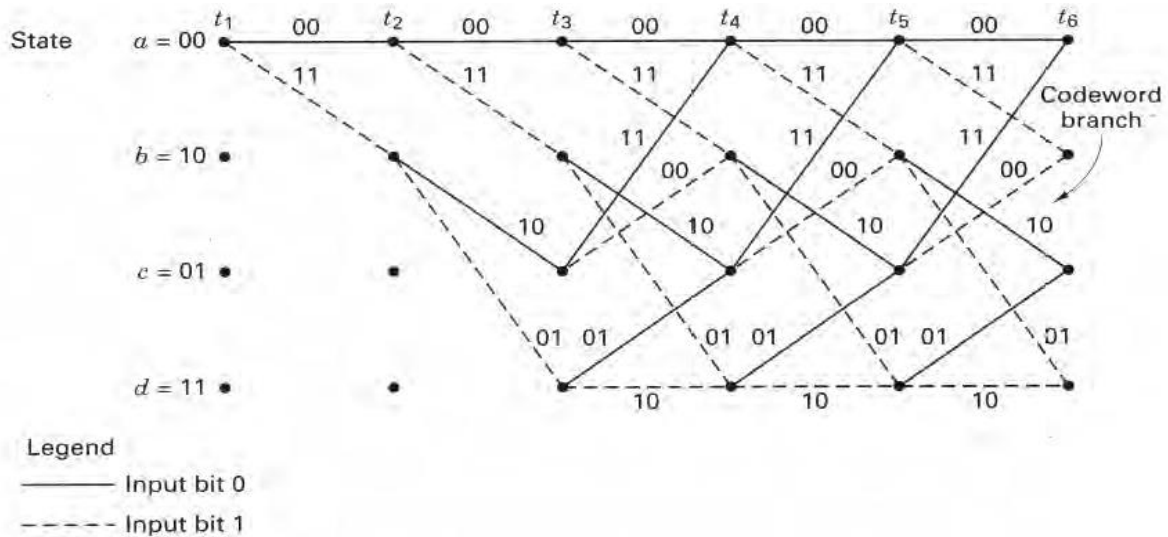
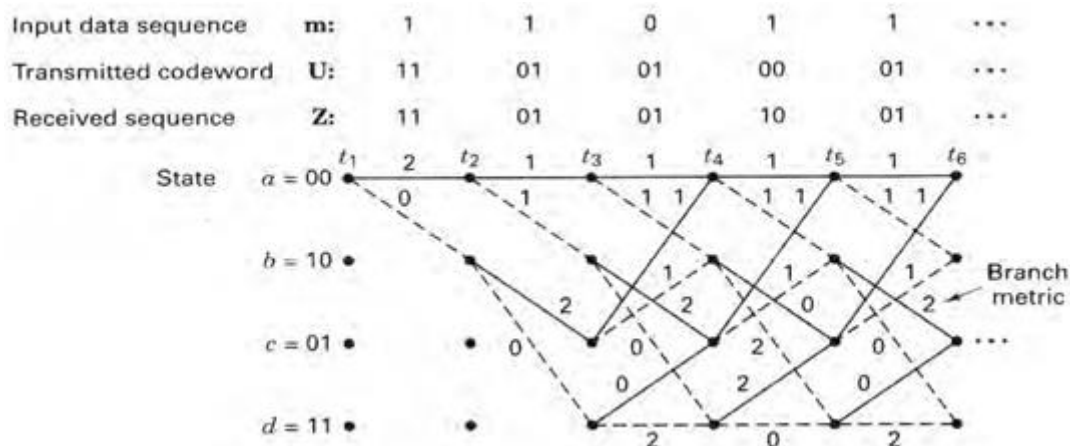


Fig (j)

the fixed structure prevails after depth K is reached. At this point and there-after, each of the states can be entered from either of two preceding states. Also, each of the states can transition to one of two states. Of the two outgoing branches, one corresponds to an input bit zero and the other corresponds to an input bit one. On Fig (j) the output branch words corresponding to the state transitions appear as labels on the trellis branches.

Hard decision Viterbi Convolution Decoding:

For simplicity, a BSC is assumed; thus Hamming distance is a proper distance measure. The encoder for this example is shown in Fig (e) and the encoder trellis diagram is shown in Fig (i). A similar trellis can be used to represent the decoder, as shown in Fig (k).

Fig (k): Decoder trellis diagram (rate $\frac{1}{2}$, $K = 3$)

We start at time t_1 in the 00 state. Since in this example there are only two possible transitions leaving any state, not all the branches need to be shown initially. The full trellis structure evolves after time after t_3 . The basic idea behind the decoding procedure can best be understood by examining the encoder trellis in concert with the Fig (k) decoder trellis. For the decoder trellis it is convenient at each time interval, to label each branch with the hamming distance between the received code symbols and the branch word corresponding to the same branch from the encoder trellis.

The example in Fig (k) shows a message sequence $\mathbf{m}$, the corresponding code sequence $\mathbf{U}$, and a noise corrupted received sequence $\mathbf{Z} = 11\ 01\ 01\ 10\ 01$. The branch words seen on the encoder trellis branches characterize the encoder in Fig (e) and are known a priori to both the encoder and decoder. These encoder branch words are the code symbols that would be expected to come from the encoder output as a result of each of the state transitions. The labels on the decoder trellis branches are accumulated by the decoder on the fly. That is, as the code symbols are received, each branch of the decoder trellis is labeled with a metric of similarity (Hamming distance) between the received code symbols and each of the branch words for that time interval.

From the received sequence $\mathbf{Z}$ shown in Fig (k), we see that the code symbols received at (following) time t_1 are 11. In order to label the decoder branches at (departing) time t_1 with the appropriate Hamming distance metric, we look at the Fig (j) encoder trellis. Here we see that a state $00 \rightarrow 00$ transition yields an output branch word of 00. But we received 11. Therefore, on the decoder trellis we labeled the $00 \rightarrow 00$ state transition with Hamming distance between them, namely 2.

Looking at the encoder trellis again, we see that a state $00 \rightarrow 10$ transition yields an output branch word of 11, which corresponds exactly with the code symbols we received at time t_1 . Therefore, on the decoder trellis, we label the state $00 \rightarrow 10$ transition with a Hamming distance of 0.

In summary, the metric entered on a decoder trellis branch represents the difference (distance) between what was received and what “should have been” received had the branch word associated with that branch been transmitted. In effect, these metric describe a correlation like measure between a received branch word and each of the candidate branch words. We continue labeling the decoder trellis branches in this way as the symbols are received at each time t_1 . The decoding algorithm uses these Hamming distance metrics to find the most likely (minimum distance) path through the trellis.

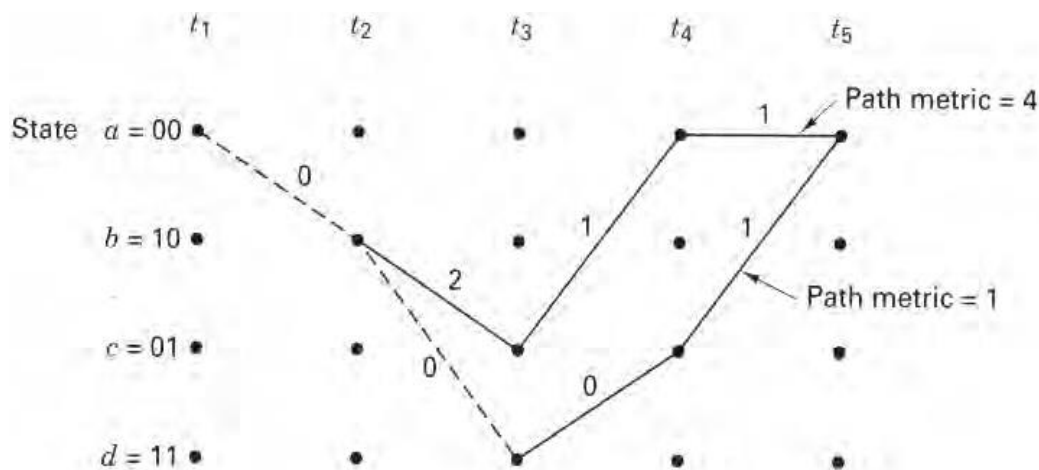


Fig (l): Path metrics for two merging path

The basis of Viterbi decoding is the following observation: If any two paths in the trellis merge to a single state, one of them can always be eliminated in the search for an optimum path. For example, Fig (l) shows two paths merging at time t_5 to state 00. Let us define the cumulative hamming path metric of a given path at time t_i as the sum of the branch Hamming distance metrics along that path up to time t_i .

In Fig (l) the upper path has metric 4; the lower has metric 1. The upper path cannot be a portion of the optimum path because the lower path, which enters the same state, has a lower metric. This observation holds because of the Markov nature of the encoder state: The present state summarizes the encoder history in the sense that previous states cannot affect future states or future states or future states or future output branches.

At each time t_i there are 2^{K-1} states in the trellis, where K is the constraint length, and each state can be entered by means of two paths. Viterbi decoding consists of computing the metrics for the two paths entering each state and eliminating one of them. This computation is done for each of the 2^{K-1} states or nodes at time t_i ; then the decoder moves to time t_{i+1} and repeats the process. At a given time, the winning path metric for each state is designated as the state metric for that state at that time.

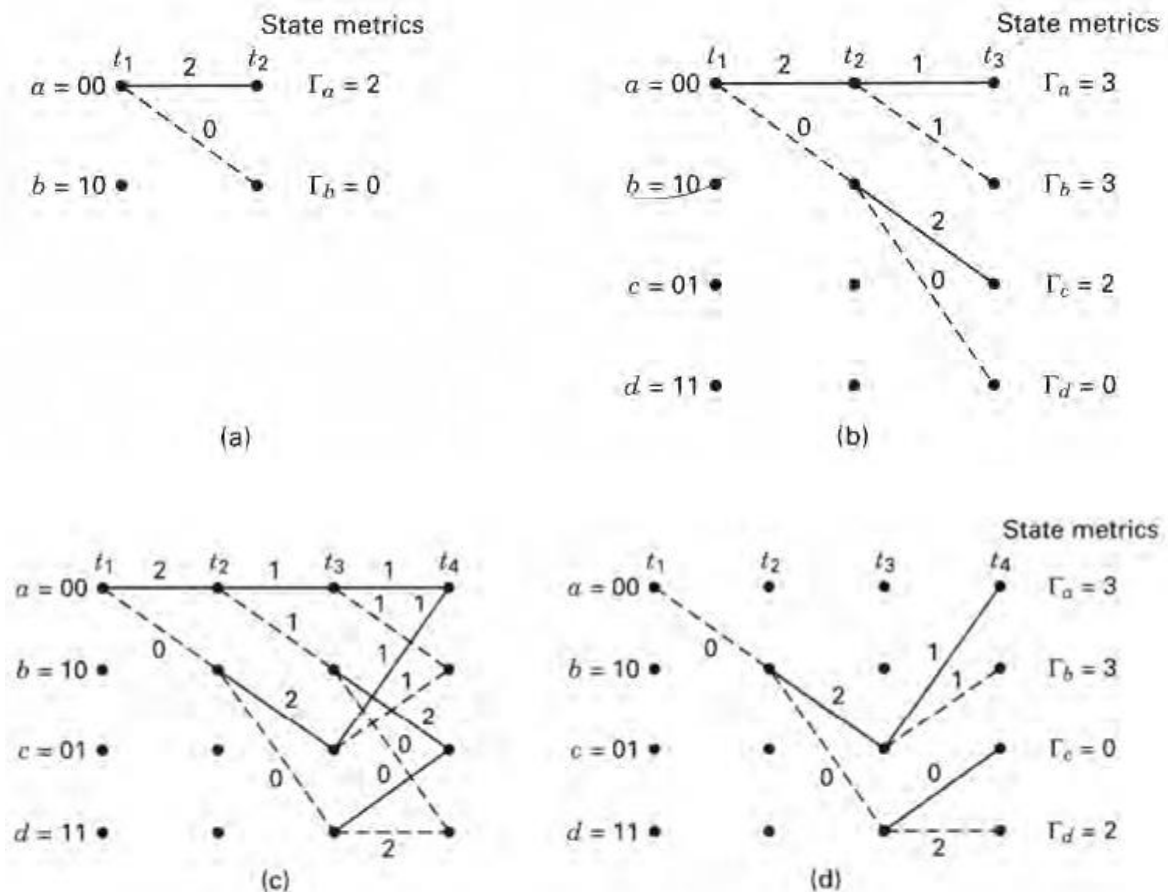


Fig (m) Selection of survivor paths, (a) Survivors at t_2 . (b) Survivors at t_3 . (c) Metric comparisons at t_4 . (d) Survivors at t_4 . (e) Metric comparisons at t_5 . (f) Survivors at t_5 . (g) Metric comparisons at t_6 . (h) Survivors at t_6 .

The first few steps in our decoding example are as follows (see Fig (j)). Assume that the input data sequence $\mathbf{m}$, code word $\mathbf{U}$, and the received sequence $\mathbf{Z}$ are as shown in Fig (m). (This assumption is not necessary in practice, but simplifies the explanation).

At time t_1 the received code symbols are 11. From state 00 the only possible state transitions are to state 00 or state 10 as shown (a) of Fig (1). State $00 \rightarrow 00$ transition has branch metric 2; state $00 \rightarrow 10$ transition has branch metric 0.

At time t_2 there are two possible branches leaving each state as shown in (b) of Fig (m). The cumulative metrics of these branches are labeled state metrics $\Gamma_a, \Gamma_b, \Gamma_c$ and Γ_d , corresponding to the terminating state.

At time t_3 in (c) of Fig (m) there are again two branches diverging from each state. As a result, there are two paths entering each state at time t_4 . One path entering each state can be eliminated, namely, the one having the larger cumulative path metric. If the metrics of two entering paths be of equal value, one path is chosen for elimination by using an arbitrary rule. The surviving path into each state is shown in (d) of Fig (m). At this point in the decoding process, there is only a single surviving path, termed the common stem, between times t_1 and t_2 . Therefore, the decoder can now decide that the state transition which occurred between t_1 and t_2 was 00 10. Since this transition is produced by an input bit one, the decoder outputs a one as the first decoded bit.

Here we can see how the decoding of the surviving branch is facilitated by having drawn the trellis branches with solid lines for input zeros and dashed lines for input ones. Note that the first bit was not decoded until the path metric computation had proceeded to a much greater depth into the trellis. For a typical decoder implementation, this represents a decoding delay which can be as much as five times the constraint length in bits.

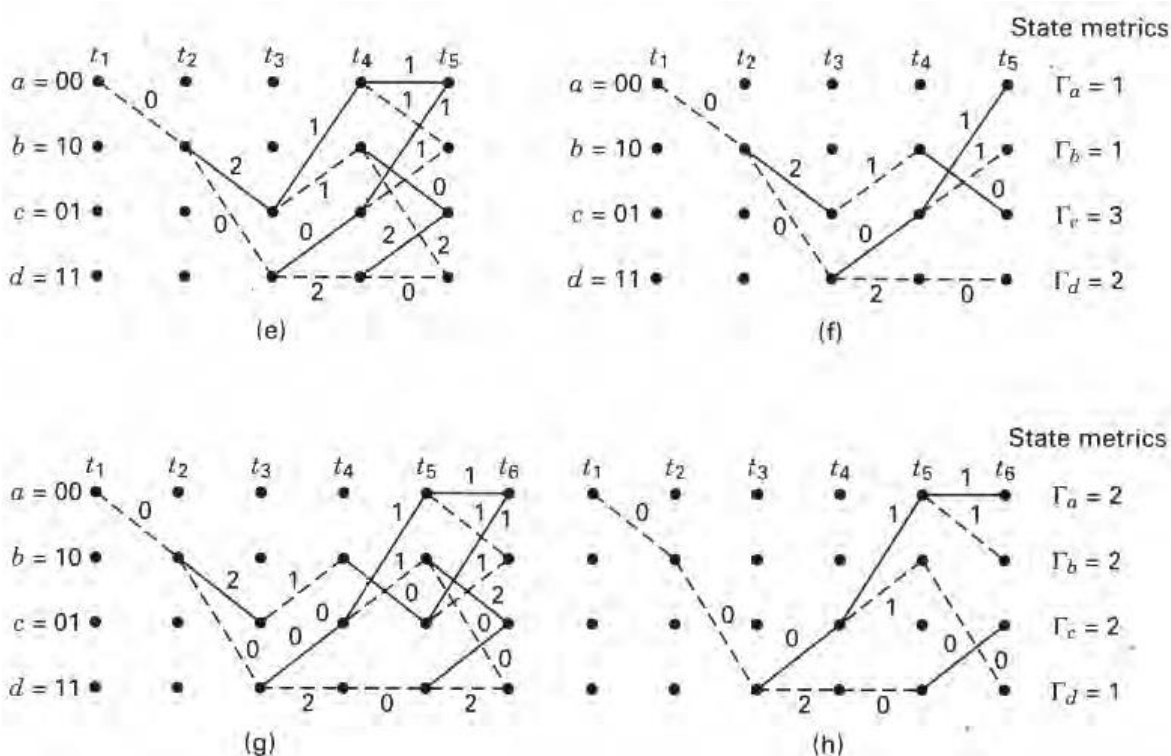


Fig (m): Continued

At each succeeding step in the decoding process, there will always be two possible paths entering each state; one of the two will be eliminated by comparing the path metrics.

(e) of Fig (m) shows the next step in the decoding process. Again at time t_5 there are two paths entering each state, and one of each pair can be eliminated. (f) of Fig (m) shows the survivors at time t_5 . Notice that in our example we cannot yet make a decision on the second input data bit because there still are two paths leaving the state 10 node at time t_2 . At time t_6 (g) of Fig (m) we again see the pattern of remerging paths, and in (h) of Fig (m) we see the survivors at time t_6 . Also in (h) of Fig (m) the decoder output one as the second

bit, corresponding to the single surviving path between t_2 and t_3 . The decoder continues in this way to advance deeper into the trellis and to make decisions on the input data bits by eliminating all paths but one.

Pruning the trellis (as paths remerge) guarantees that there are never more paths than there are states. For Example, verify that after each pruning in (b), (d), (f), (h) Fig (m) there are 4 paths. Compare this to attempting a “brute force” maximum-likelihood sequence estimation without using the Viterbi algorithm. In that case, the number of possible paths (representing possible sequences) is an exponential function of sequence length. For a binary code word sequence that has a length of L branch word, there are 2^L possible sequences.

EQUIPMENTS:

- DCL-CONV Kit
- Patch Chords
- Power supply
- DSO

PROCEDURE:

1. Refer to the Fig. 1.1 and carry out the following connections and switch settings.
 2. Connect power supply in proper polarity to the kit **DCL-CONV** and switch it on.
 3. Keep the Data clock select switch **SW2** towards slow position.
 4. Select data pattern using select switch **SW1** in the **Data Generator** block.
 5. Connect **SERIALDATA** generated on board to **DATA IN** of **CONVOLUTION ENCODER**.
 6. Observe **RDY1** pin, convolutionally encoded data will be observed at **OUT1** and **OUT2** post. The convolutionally encoded data are valid from the instant when **RDY1** goes high.
 7. Connect **OUT1** and **OUT2** post of **Convolution Encoder** block **IN1** and **IN2** of **Hard Decision Viterbi Decoder** block.
 8. Observe the decoded data at the **DATA OUT1** post of **Hard Decision Viterbi Decoder** block.
 9. Repeat the procedure by keeping the data clock select switch towards fast position.
- **Note1: The reason for the provision of data clock with high and low frequency, using data clock select switch is to observe the encoded and decoded data in a slow as well as fast mode.**

- **Note2:** It is advised to observe the decoded data i.e. output of the Hard decision Viterbi Decoder in fast mode because it takes approximately four minutes by the Viterbi decoder to decode the data since the operating frequency is very low.

OBSERVATION:

Observe the following signal on the oscilloscope and you can verify it by solving, for the selected data pattern, using the steps given in the theory of this experiment.

Also note that in the theory we have taken input data of only five bits for simplicity but DCL-CONV supports 8 bit input data pattern which is continuous in nature, so while solving for different data pattern always take two to three samples of 8 bit input data pattern.

1. **SERIAL DATA** with respect to **DATA CLK**. (FIG. 1.2 (a))
2. **OUT1** with respect to **RDY1**. (FIG. 1.2 (b))
3. **OUT 2** with respect to **RDY1**. (FIG. 1.2 (c))
4. **OUT 1** with respect to **OUT 2**. (FIG. 1.2 (d))
5. **DATA OUT1** with respect to **SERIAL DATA**. (FIG. 1.2 (e))

CAPTURED WAVEFORM:

CH 1: DATA CLK (256 KHz OR 1 Hz) & CH 2: SERIAL DATA (00011011)

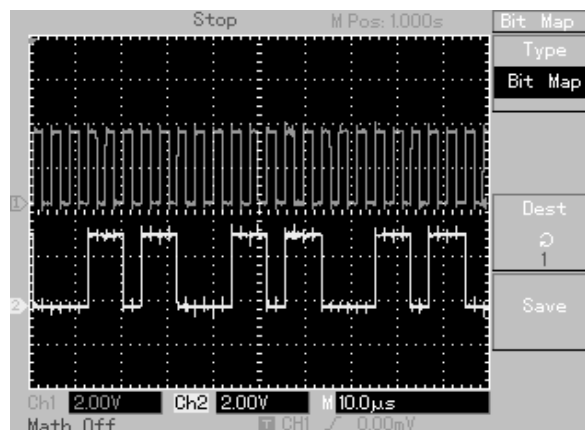
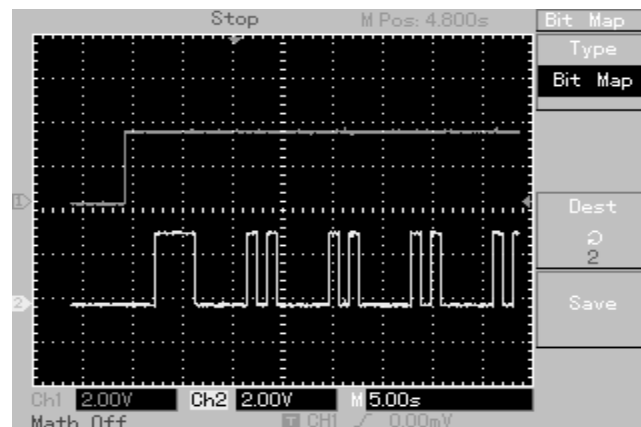
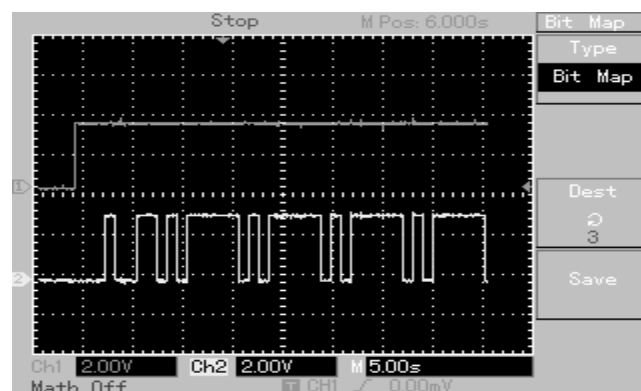
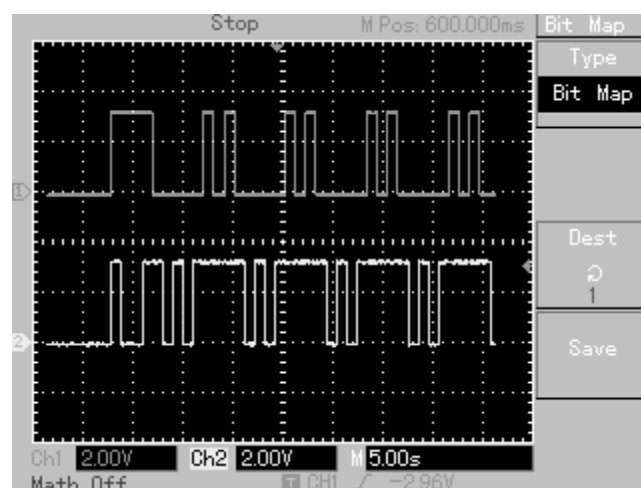
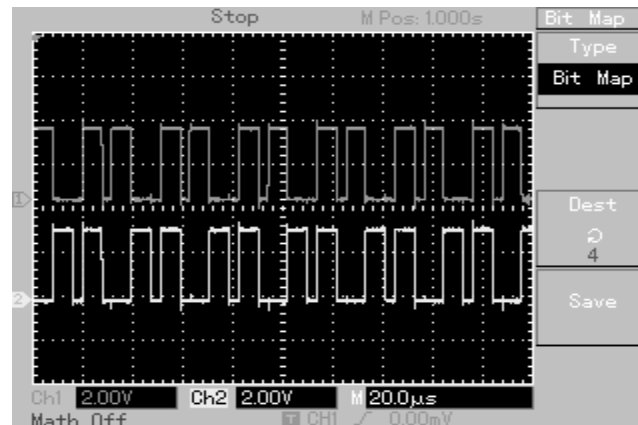


FIG. 1.2 (a)

CH 1: RDY1 & CH 2: OUT1**FIG. 1.2 (b)****CH 1: RDY1 & CH 2: OUT2****FIG. 1.2 (c)****CH 1: OUT1 & CH 2: OUT2****FIG. 1.2 (d)**

CH 1: SERIAL DATA& CH 2: DATA OUT1**FIG. 1.2 (e)****CONCLUSION:**

Thus we have studied convolution encoding and hard decision viterbi decoding technique of serial data and also observed how a serial data is convolutionally encoded and when it is passed through an error free medium how it is decoded using Hard Decision Viterbi Decoder.

EXPERIMENT NO. 2

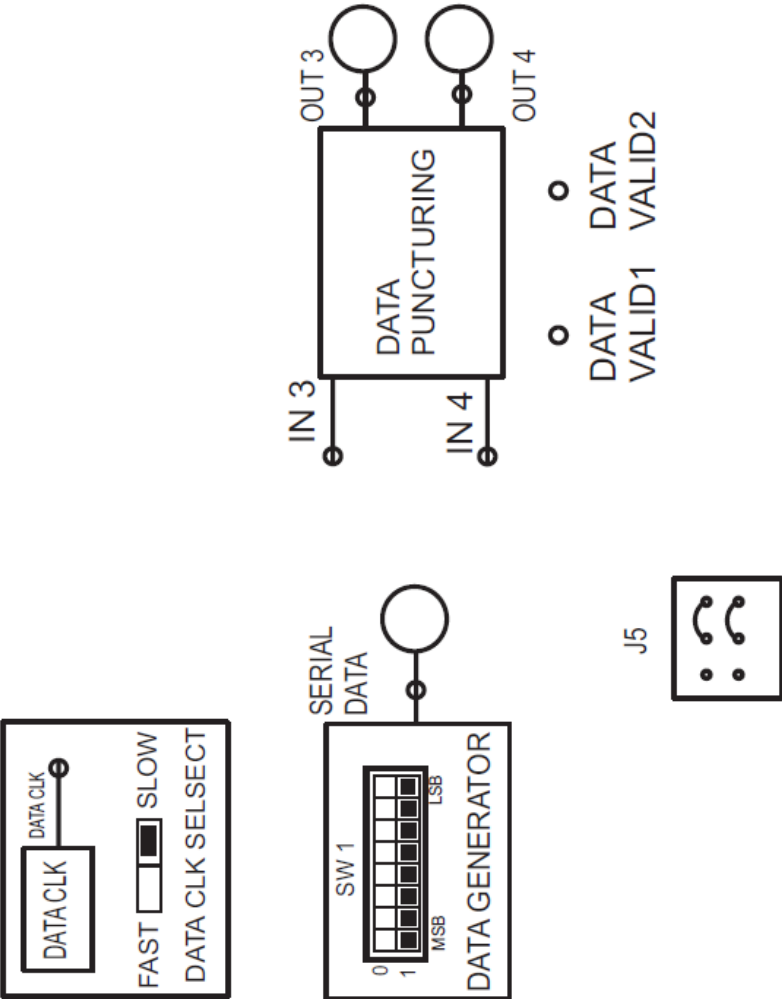


FIG 2.1 BLOCK DIAGRAM FOR
STUDY OF CONVOLUTION ENCODER USING
PUNCTURING FOR K=7, RATE = 1/2

EXPERIMENT NO: 2

NAME:

Convolution Encoding using Puncturing for $K=7$ and Rate = $3/4$.

OBJECTIVE:

Study of Convolution Encoding using Puncturing for $K=7$ and Rate = $3/4$.

THEORY:

The entire convolutional encoding technique is described in experiment number one of DCL-CONV, here in this experiment we will focus on the concept of puncturing.

The intent of this experiment is to help clarify the terms used to define the punctured convolutional encoding as well as to explain how convolutional encoding along with puncturing takes place theoretically and to observe and verify the results practically.

Fig (a) shows the block diagram of convolutional encoder along with puncturing.

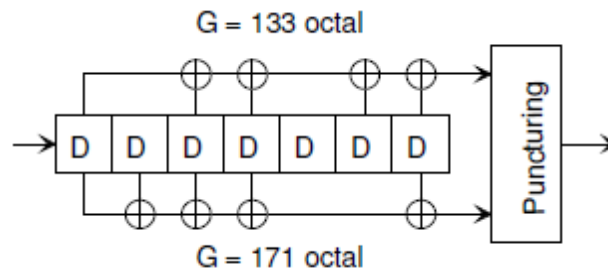


Fig (a)

Puncturing:

After convolutional encoding, some of the encoded symbols can be selectively removed before transmission. This process, called "puncturing," is a data compression method used to reduce the number of bits transmitted.

Puncturing is a technique used to make an m/n rate code from a "basic" rate code. It is achieved by deletion of some bits in the encoder output. Bits are deleted according to a puncturing matrix. A "1" in the puncturing matrix means the corresponding symbol is kept in the output stream, while a "0" means the symbol in that position is removed.

Punctured convolutional codes:

Punctured convolutional codes are derived from ordinary codes by dropping some output bits (i.e. not sending them to a channel). The resulting code has a higher rate (less redundancy) but lower correcting capability. The order of dropping is defined by a

puncturing matrix. The number of rows in a puncturing matrix is equal to the number of output polynomials. The elements of this matrix are 1's (which means that we transmit a given bit) and 0's (which means that we drop it). The puncturing matrix is applied to the output stream cyclically. The standard puncturing matrices for the rate $\frac{1}{2}$, $K = 7$ (INTELSAT) convolutional codes are:

Code rate	Generator polynomial	Puncturing Matrix
Rate 2/3	$G_0 = 133_8$ $G_1 = 171_8$	11 10
Rate 3/4	$G_0 = 133_8$ $G_1 = 171_8$	110 101
Rate 5/6	$G_0 = 133_8$ $G_1 = 171_8$	11010 10101
Rate 7/8	$G_0 = 133_8$ $G_1 = 171_8$	1111010 1000101

Fig (b): $R = \frac{1}{2}$, $K=7$ (Intelsat) puncturing matrix

For example, consider a puncturing matrix for the rate $\frac{2}{3}$. It means that we will transmit only every bit from the first polynomial, but every second bit from the second polynomial.

Fig (c) given below shows an example of puncturing process, i.e. how a basic rate code $\frac{1}{2}$ is converted into code rate $\frac{3}{4}$. When input data is applied to convolution encoder we get convolution codes. After coding, the convolution codes are punctured using the puncturing matrix 101 and 110 where "1" in the puncturing matrix means the corresponding symbol is kept in the output stream, while a "0" means the symbol in that position is removed. Hence a_1 , a_4 , b_2 and b_5 are removed from the final punctured output of Fig (c).

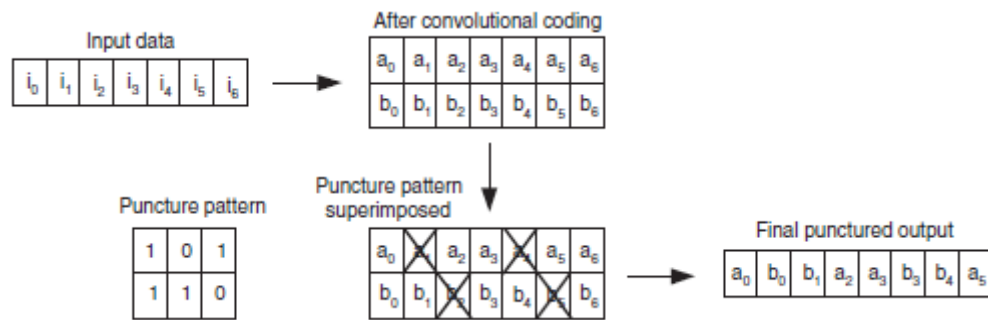


Fig (c): Puncturing process

Let us take the example where we have to generate a code rate of $\frac{3}{4}$, by using a basic encoder with code rate $\frac{1}{2}$. For this let us take a 8-bit serial input data “11111111” applied to a basic convolution encoder of $K=7$ and $R=1/2$ with generator polynomial:

$$G_0(x) = 1+x^2+x^3+x^5+x^6$$

$$G_1(x) = 1+x+x^2+x^3+x^6$$

We can approach the encoder in terms of its impulse response i.e. the response of the encoder to a single “one” bit that moves through it. Consider the contents of the register in Fig (a)

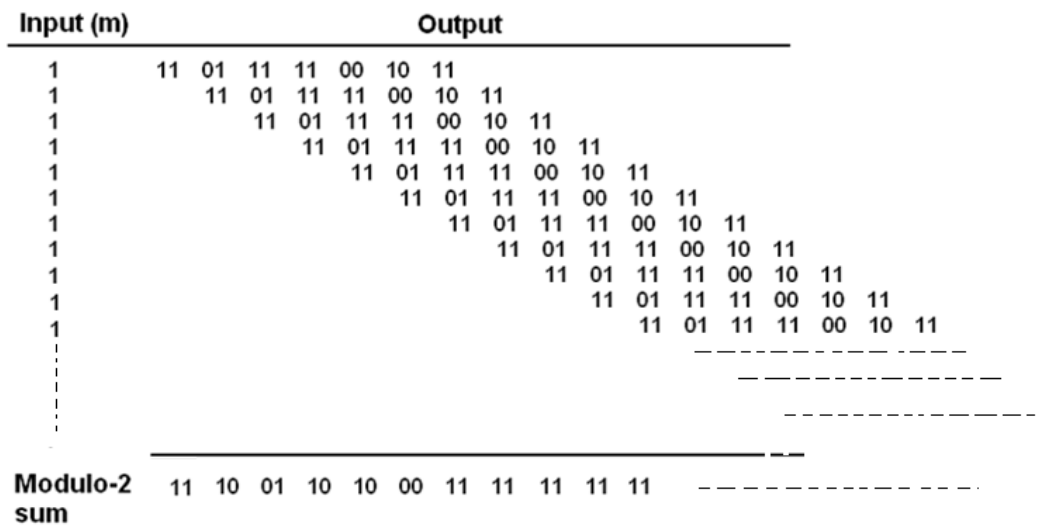
Register contents	Branch word	
	U1	U2
1 0 0 0 0 0 0	1	1
0 1 0 0 0 0 0	0	1
0 0 1 0 0 0 0	1	1
0 0 0 1 0 0 0	1	1
0 0 0 0 1 0 0	0	0
0 0 0 0 0 1 0	1	0
0 0 0 0 0 0 1	1	1

Fig (d)

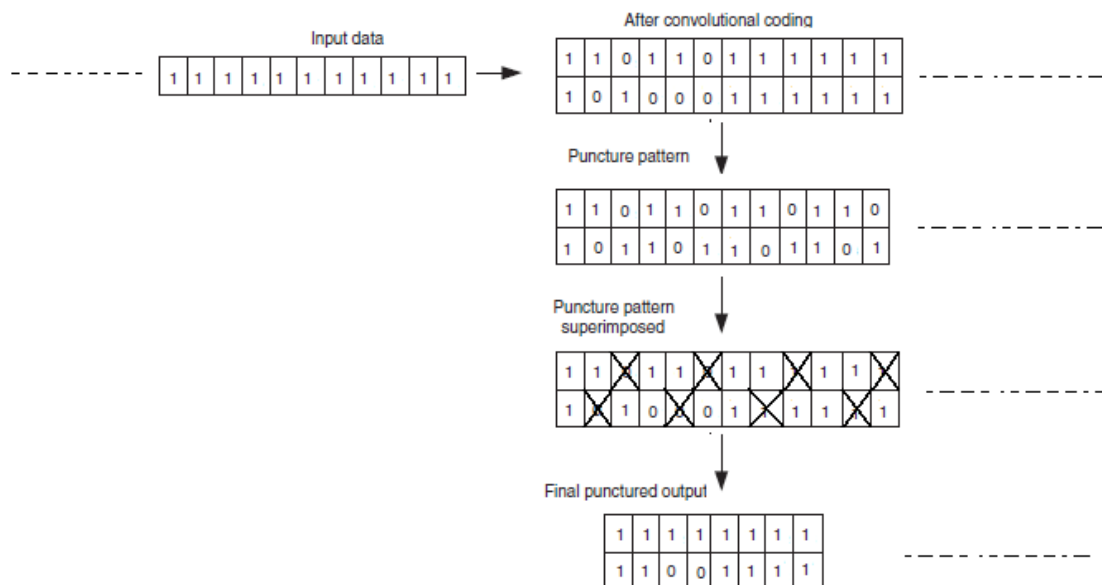
Input sequence: 1 0 0 0 0 0 0
 Output sequence: 11 01 11 11 00 10 11

The output sequence for the input “one” is called the impulse response of the encoder. Then for the input sequence $m = 1 1 1 1 1 1 1 1 1 1 1 \dots$, the output may be found by the superposition or the linear addition of the time shifted input “impulses” as follows:

Note: DCL-CON V supports 8-bit input data. But this 8 bit input data is repeating continuously. Hence to avoid confusion we are taking 11 bits in the example in which input data after eighth bit are repetition of data pattern.



Now to get the rate $\frac{3}{4}$, the output of the convolution encoder of rate $= \frac{1}{2}$ and $K = 7$, is punctured by using the puncturing matrix for rate $= \frac{3}{4}$ from Fig (b). A “1” in the puncturing matrix means the corresponding symbol is kept in the output stream, while a “0” means the symbol in that position is removed as shown in Fig (f). Thus we get the final punctured output as “111110101111111 -----”.



EQUIPMENTS:

- DCL-CONV Kit
- Patch Chords
- Power supply
- DSO.

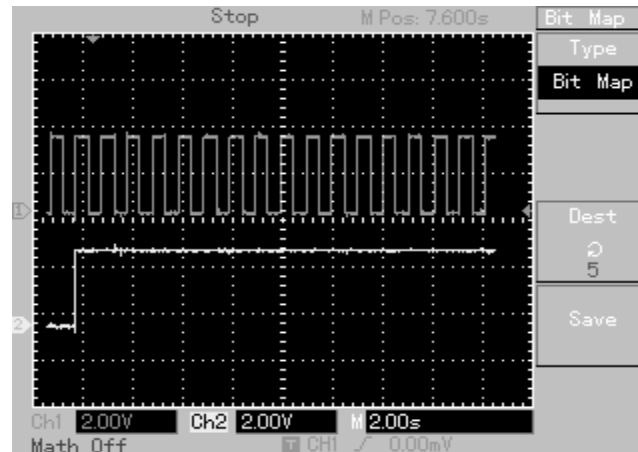
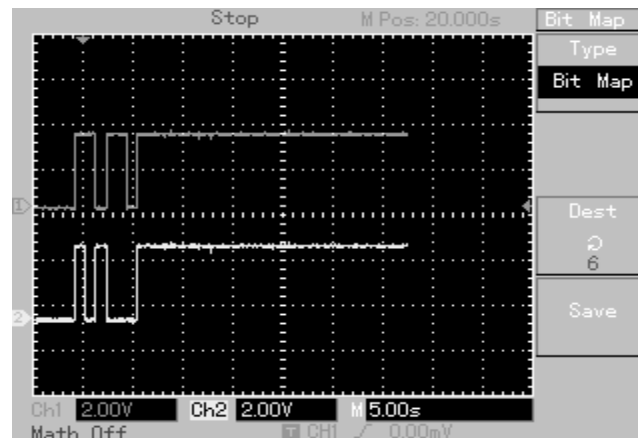
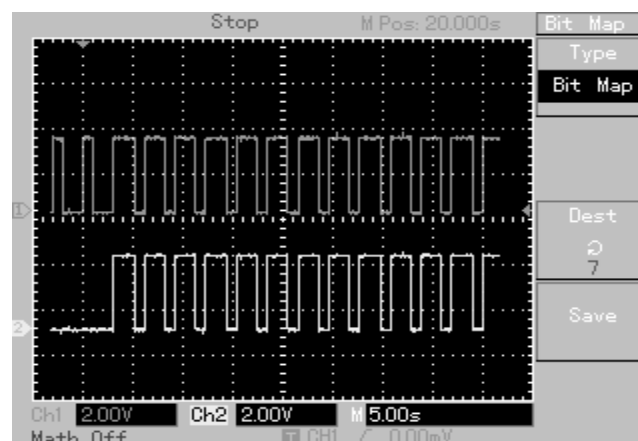
PROCEDURE:

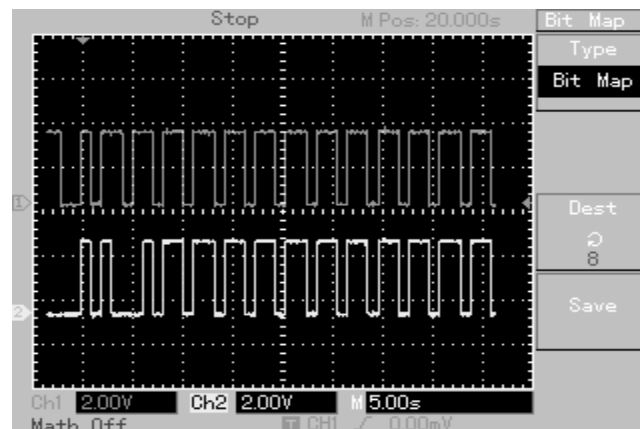
1. Refer to the Fig. 2.1 and carry out the following connections and **data clock switch** setting **SW2** in slow mode.
2. Connect power supply in proper polarity to the kit **DCL-CONV** and switch it on.
3. Select a data pattern using switch **SW1** of **data generator block**.
4. Keep the jumper **J5** settings in the **puncturing** block as shown in Fig 2.1. Thus the Serial Data generated is connected to the input of convolution encoder and we can observe convolutionally encoded data at **IN3** and **IN4** post of **data puncturing** block.
5. Observe the convolutionally encoded data at **IN3** and **IN4** of **Data Puncturing** block.
6. Observe the punctured convolutional code at **OUT3** with **DATA VALID1** and **OUT4** with **DATA VALID2** of **data puncturing** block. Note that a logic high on **DATA VALID1** and **DATA VALID2** indicates that there is a valid punctured data on **OUT3** and **OUT4** respectively.
7. Repeat the procedure by keeping the **data clock select** switch **SW2** to fast mode.
8. Observe various waveforms as mentioned below.

OBSERVATION:

Observe the following signal on the oscilloscope and you can verify it by solving, for the selected data pattern, using the steps given in the theory of this experiment.

- | | |
|---|----------------|
| 1. SERIAL DATA with respect to DATA CLK . | (FIG. 2.3 (a)) |
| 2. IN3 and IN4 . | (FIG. 2.3 (b)) |
| 3. OUT3 with respect to DATA VALID1 . | (FIG. 2.3 (c)) |
| 4. OUT4 with respect to DATA VALID2 . | (FIG. 2.3 (d)) |

CAPTURED WAVEFORM:**CH 1: DATA CLK (256 KHz OR 1 Hz) & CH 2: SERIAL DATA (11111111)****FIG. 2.3 (a)****CH 1: IN3 & CH 2: IN4****FIG. 2.3 (b)****CH 1: DATA VALID1 & CH 2: OUT3****FIG. 2.3 (c)**

CH 1: DATA VALID2 & CH 2: OUT4**FIG. 2.3 (d)****CONCLUSION:**

Thus we have studied convolution encoding using puncturing and also observed how a serial data is convolutionally encoded and then punctured using puncturing matrix.

EXPERIMENT NO. 3

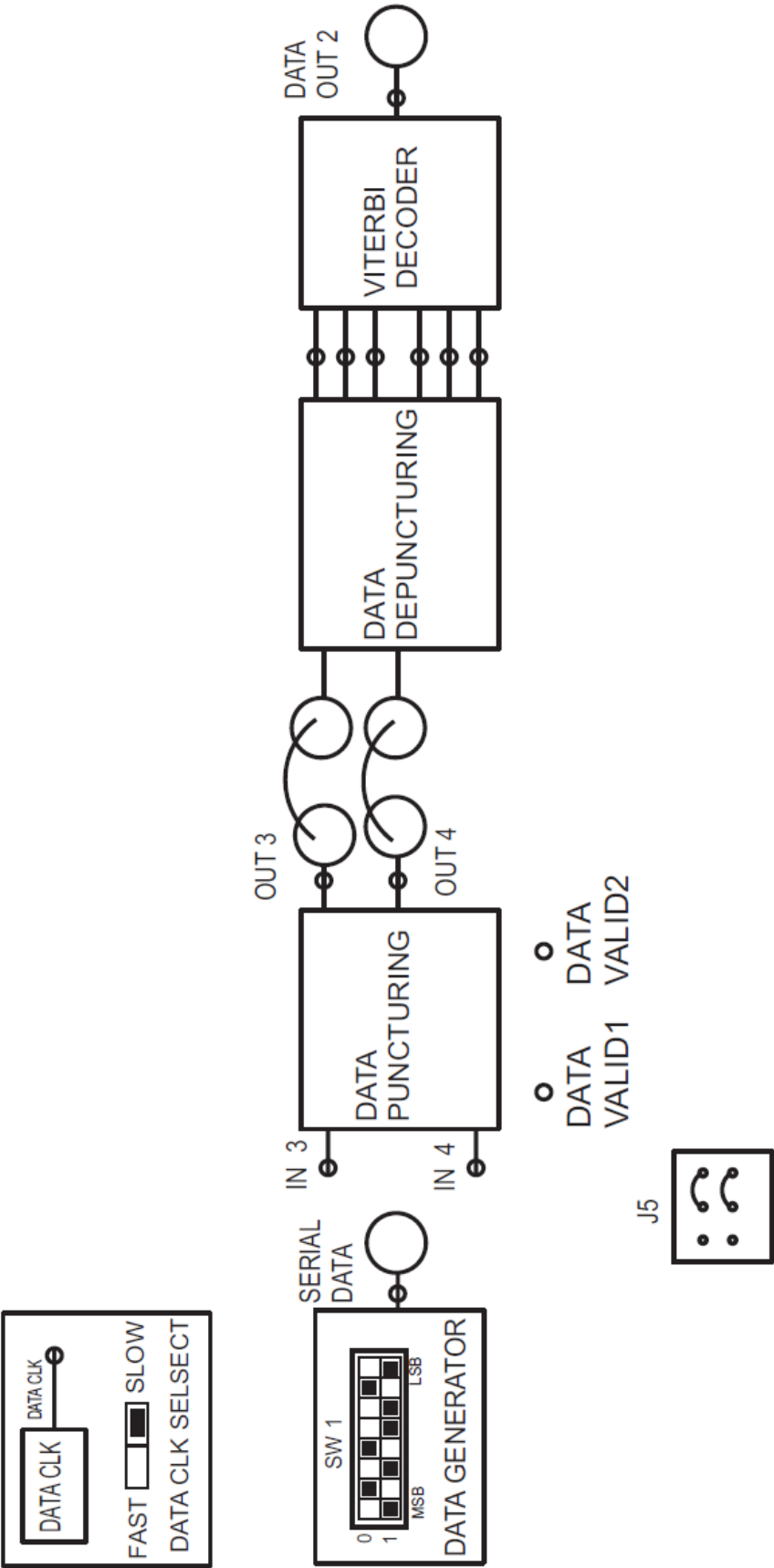


FIG 3.1 BLOCK DIAGRAM FOR
STUDY OF CONVOLUTION ENCODER USING PUNCTURING AND
SOFT DECISION VITERBI DECODER USING DEPUNCTURING FOR
 $K=7$, RATE = $\frac{1}{2}$

EXPERIMENT NO: 3

NAME:

Convolution Encoding using Puncturing and Soft decision Viterbi Decoding using Depuncturing.

OBJECTIVE:

Study of Convolution Encoding using Puncturing and Soft decision Viterbi Decoding using Depuncturing for $K=7$ and Rate = $3/4$.

THEORY:

The concept of Convolution encoding using puncturing has been explained in experiment two of DCL-CONV, refer the theory part to understand it in detail. In this experiment we are going to focus on depuncturing and soft decision viterbi decoding.

Depuncturing:

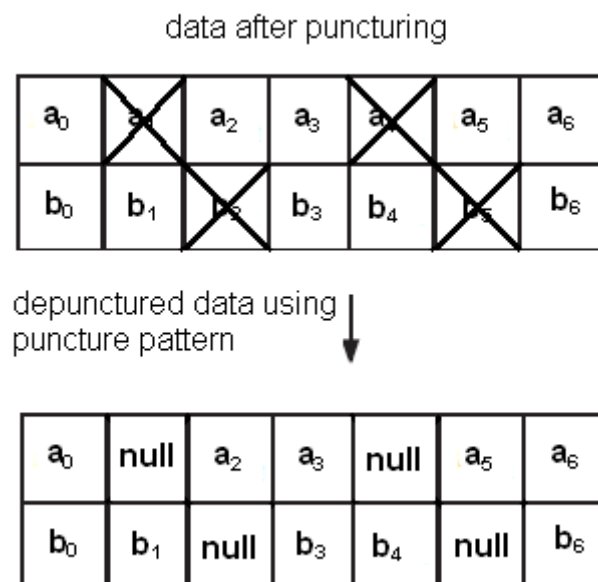


Fig (a)

Fig (a) shows depuncturing of punctured data. If puncturing is employed in the encoder, the decoder will have to “depuncture” the data before decoding. Depuncturing is done by inserting NULL symbols for the punctured symbols. NULL symbols are equidistant from both ‘0’ and ‘1’. In order to decode a punctured convolutional code, one need first to insert erasure marks to the places where punctured symbols should be. This task is performed by a depuncturer. A pair of binary strings, called a “puncture pattern,” is used to identify punctured symbols. A “1” in a pattern means the corresponding symbol was not punctured in the encoder, while a “0” means the symbol has been punctured.

Note that the output of the depuncturer in DCL-CONV, is in signed magnitude form because the soft decision viterbi decoder supports either sign-magnitude or unsigned offset format. The concept of sign-magnitude and unsigned offset format is explained below.

Data Format:

The soft decision viterbi decoder supports two commonly used binary representations, namely, sign-magnitude and unsigned offset, for soft decision data. In sign-magnitude representation, the most significant bit is a sign bit and the rest of the bits represent the magnitude. The most positive number corresponds to strong logic zero and other positive numbers are weak logic zeros. The most negative number corresponds to strong logic one and other negative numbers are weak logic ones.

In unsigned offset representation, there is no sign bit in the number and all numbers are treated positive. The smallest number (all zeros) corresponds to strong logic zero and the biggest number (all ones) corresponds to logic one. The smaller numbers counting up from zero are progressively weaker logic zeros and bigger numbers counting down from the biggest number are progressively weaker logic ones.

Interpretation of Signed and Unsigned Data

Signed Binary			Unsigned Offset		
Data	Interpretation		Data	Interpretation	
111	-3	(strong logic 1)	111	7	(strong logic 1)
110	-2	(weaker logic 1s)	110	6	(weaker logic 1s)
101	-1		101	5	
100	-0		100	4	
000	0	(weaker logic 0s)	011	3	(weaker logic 0s)
001	1		010	2	
010	2		001	1	
011	3	(strong logic 0)	000	0	(strong logic 0)

Fig (b)

Note that for the sake of better explanation of soft decision viterbi decoder we are going to take an example, we will divide it into two sections namely Encoder and Decoder. We will revise everything in short and give major emphasis on soft decision viterbi decoder. In DCL-CONV the soft decision viterbi decoder uses input in signed magnitude format but for the sake of better explanation we are going to assume that the soft decision viterbi decoder for the example given below uses unsigned offset format.

Let us take the same example, mentioned in experiment one for explanation of convolution encoder using puncturing and soft decision viterbi decoding using depuncturing technique. The Fig (c) illustrates a (2, 1) convolutional encoder with constraint length $K = 3$. There are $n = 2$ modulo-2 adder; thus the code rate $k/n = \frac{1}{2}$.

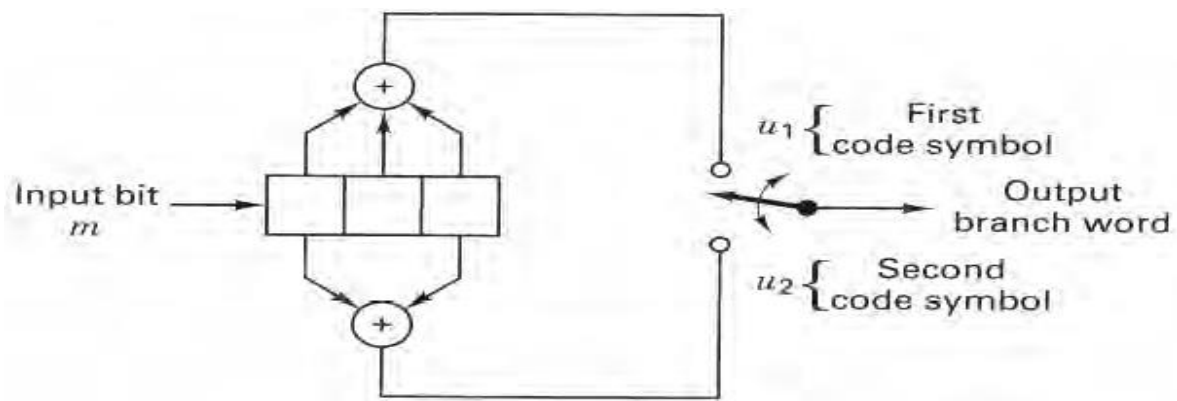


Fig (c): Convolutional Encoder (rate $\frac{1}{2}$, $K = 3$)

Encoder:

For the above stated example, we have calculated impulse response, code word, state transition table, state diagram and trellis diagram of the encoder as:

Impulse response:

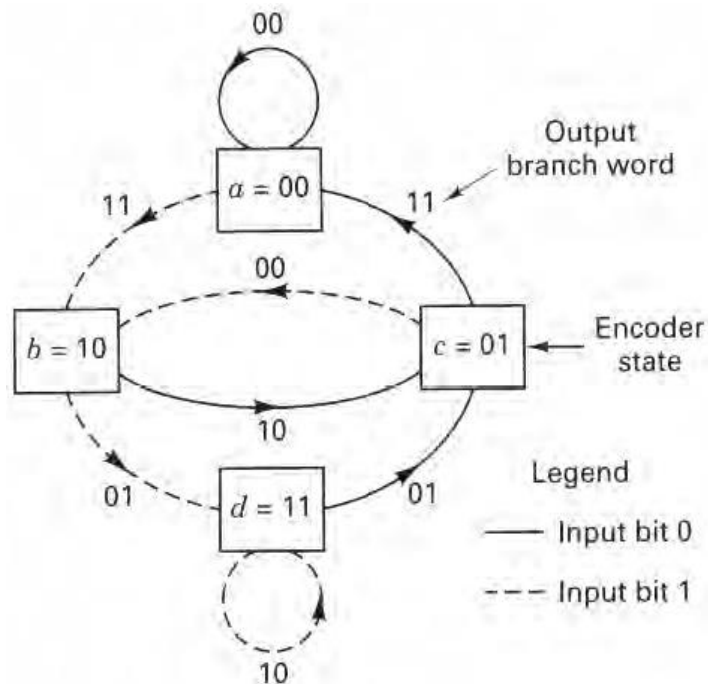
11 01 11

Code word:

11 01 01 00 01 01 11

State transition table:

Input state	Input bit	Output state	Output bit
a	0	a	00
	1	b	11
b	0	c	10
	1	d	01
c	0	a	11
	1	b	00
d	0	c	01
	1	d	10

State Diagram:**Encoder Trellis Diagram:****Puncturing:**

As we know that once the code words are generated, for data compression and to reduce the number of bits transmitted, the code words are punctured. The concept of puncturing is well explained in experiment two.

Decoder:**Depuncturing:**

Since puncturing is used on the encoder side, data depuncturing has to be done at the decoder side before decoding. Depuncturing is done by inserting NULL symbols for the punctured symbols. NULL symbols are equidistant from both '0' and '1'. In order to decode a punctured convolutional code, one needs first to insert erasure marks to the places where punctured symbols should be. This task is performed by a depuncturer. A pair of binary strings, called a "puncture pattern," is used to identify punctured symbols. A "1" in a pattern means the corresponding symbol was not punctured in the encoder, while a "0" means the symbol has been punctured.

Consider the same example that is mentioned in experiment no. 2. For the input data pattern "111111111111 -----" how the convolution encoding along with puncturing is done is well explained in that experiment. Here in this part we will discuss how

depuncturing of punctured convolution code is carried out and how depunctured data is converted into signed binary format.

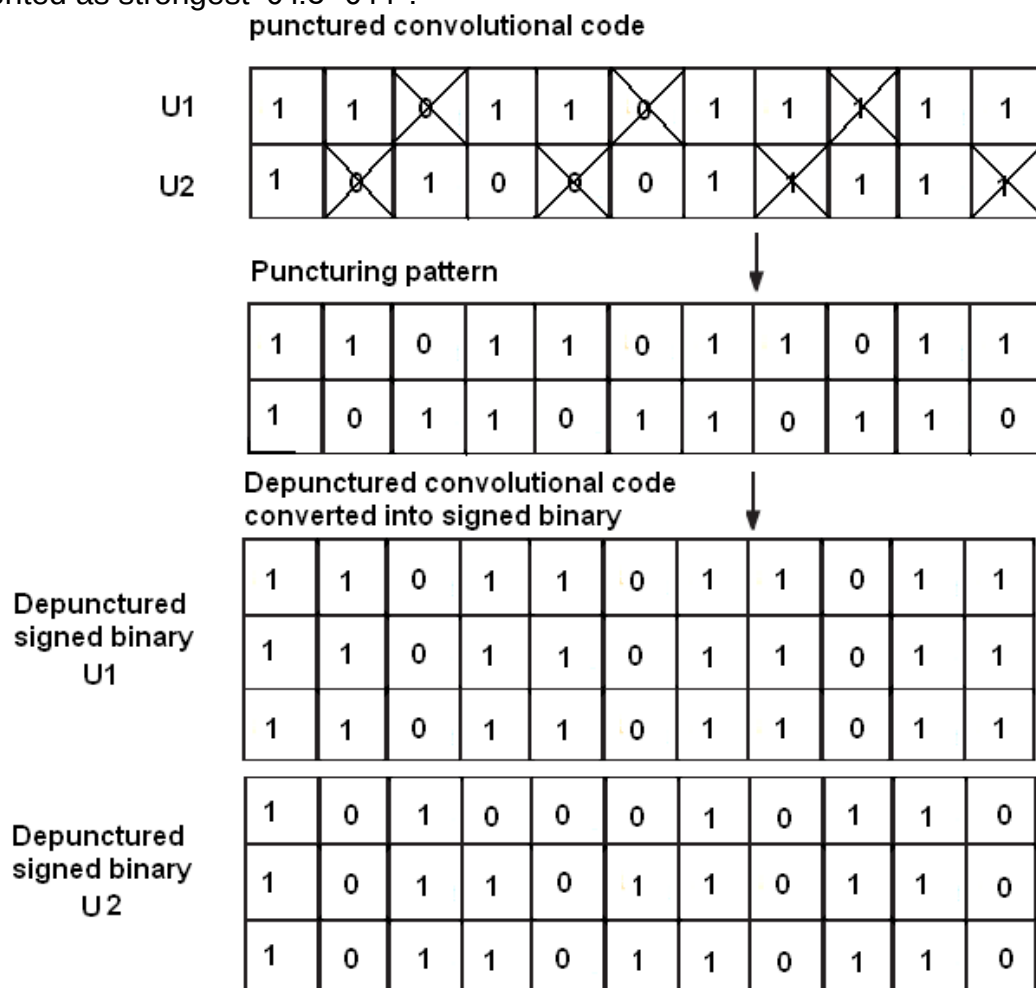
Now the output of the convolutional encoder for input data pattern "111111111111---" is,

U1 = 1 1 0 1 1 0 1 1 1 1 1

U2 = 1 0 1 0 0 0 1 1 1 1 1

During depuncturing the location of the bits which are punctured are denoted by cross in the punctured convolutional code in the figure below. At that location we insert null symbol i.e. '0' and since the null symbol is equidistant from '0' and '1' in signed binary format it is represented as weakest '0' i.e. "000" which can be seen in Depunctured signed binary U1 and Depunctured signed binary U2, in the fig below.

The location where there is '1' in the punctured convolutional code is represented as strongest '1' i.e. "111" and the location where there is '0' in the punctured convolution code is represented as strongest '0' i.e. "011".



Soft decision viterbi decoder:

Here for better understanding of soft decision decoding, let us compare it with hard decision decoding. As we know that for a rate 1/2 convolutional code system the demodulator delivers two code symbols at a time to the decoder.

For hard-decision (2-level) decoding, each pair of received code symbol can be depicted on a plane as one of the corners of a square, as shown in Fig (d). The corners are labeled

with the binary numbers (0,0), (0,1), (1,0), (1,1), representing the four possible hard decision values that the two code symbols might have.

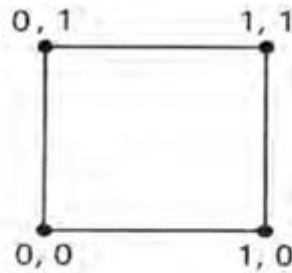


Fig (d) Hard-decision plane

For 8-level soft decision decoding, each pair of code symbols can be similarly represented on an equally spaced 8-level by 8-level plane, as a point from the set of 64 points shown in (1) of fig (e) given below. In this soft decision case the demodulator no longer delivers firm decisions; it delivers quantized noisy signals (soft decisions).

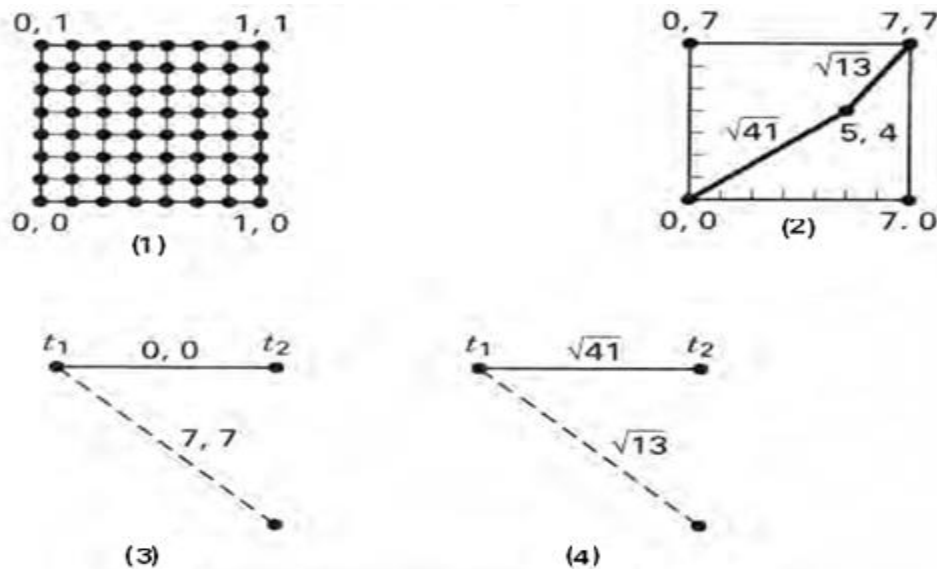
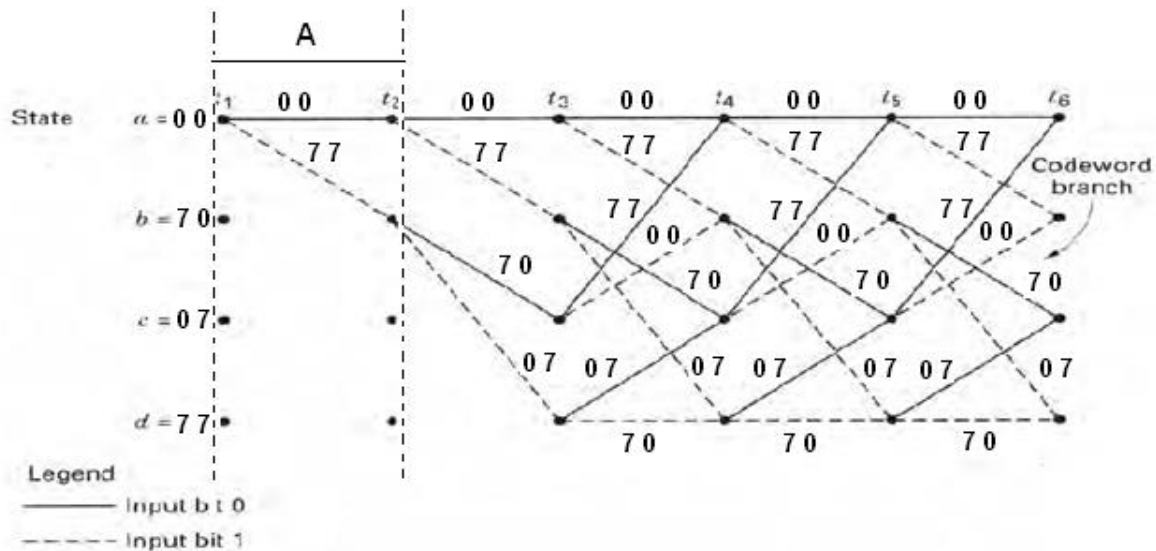


Fig (e) (1) 8-level by 8-level soft-decision plane
(2) Example of soft code symbols (3) Encoding trellis section
(4) Decoding trellis section.

The primary difference between hard decision and soft decision Viterbi decoding is that soft decision algorithm cannot use a Hamming distance metric because of its limited resolution.

A distance metric with the needed resolution Euclidean distance, and to facilitate its use, the binary numbers of 1 and 0 are transformed to octal numbers 7 and 0, respectively i.e. in unsigned offset format. This can be seen in (2) of Fig (e), where the corners of the square have been re-labeled accordingly this allows us to use a pair of integers, each in the range of 0 to 7, for describing any point in the 64-point set. Note that DCL-CONV supports signed binary format but for sake of simplicity we are using unsigned offset format in the theory.

Let us represent the encoder trellis in unsigned offset binary format i.e. the binary numbers of 1 and 0 are transformed to octal numbers 7 and 0, respectively. Fig (f) shows encoding trellis section.

Fig (f) Encoder trellis diagram (rate $\frac{1}{2}$, $K=3$).

Also shown in the (2) of Fig (e) is the point 5, 4, representing an example of a pair of noisy code-symbol values that might stem from a demodulator. Imagine that the square in (2) of Fig (e) has co-ordinates x and y . Then, what is the Euclidean distance between the noisy point 5,4 and the noiseless point 0, 0? It is $\sqrt{(5-0)^2 + (4-0)^2} = \sqrt{41}$. Similarly, if we ask what is the Euclidean distance between the noisy point 5, 4 and the noiseless point 7, 7? It is $\sqrt{(5-7)^2 + (4-7)^2} = \sqrt{13}$.

Soft-decision Viterbi decoding, for the most part proceeds in the same way as hard-decision decoding. The only difference is that Hamming distances are not used.

Consider how soft-decision decoding is performed with the use of Euclidean distances. (3) of Fig (e) shows the first section of encoding trellis, originally presented in section A denoted by dotted line of fig (f) with the branch words transformed from binary to octal.

Suppose that a pair of soft decision code symbol with values (5, 4) arrives at a decoder during the first transition interval. (4) of Fig (e) shows the first sequence of a decoding trellis. The metric ($\sqrt{41}$), representing the Euclidean distance between the arriving 5, 4 and the 0, 0 branch word, placed on the solid line. Similarly the metric ($\sqrt{13}$), representing the Euclidean distance between the arriving 5, 4 and the 7, 7 code symbols is placed on the dashed line. The rest of the task, pruning the trellis in search of a common stem proceeds in the same way as hard decision decoding.

Thus with such a metric, we would seek maximum correlation rather than minimum distance.

EQUIPMENTS

- DCL-CONV Kit.
- Patch Chords.
- Power supply.
- DSO.

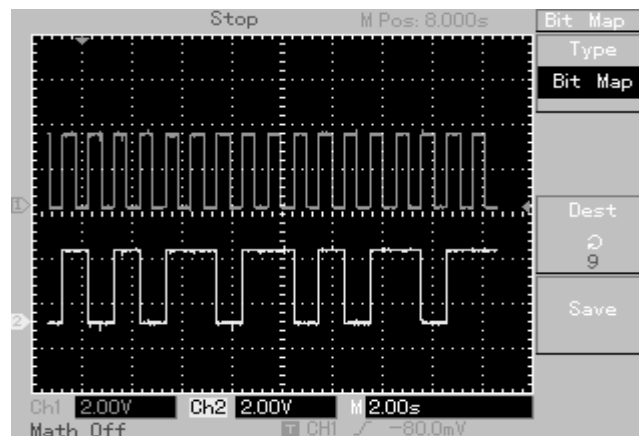
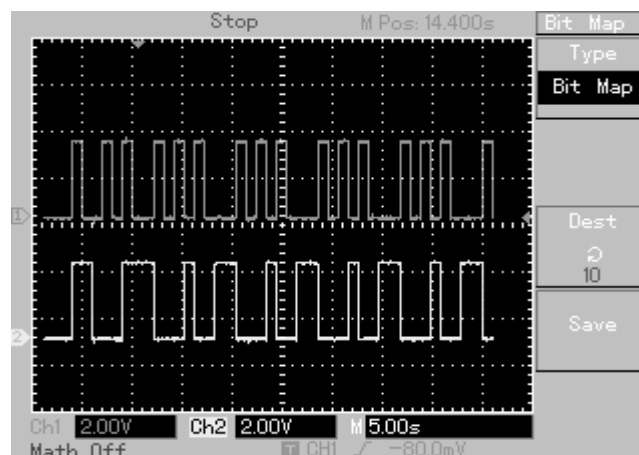
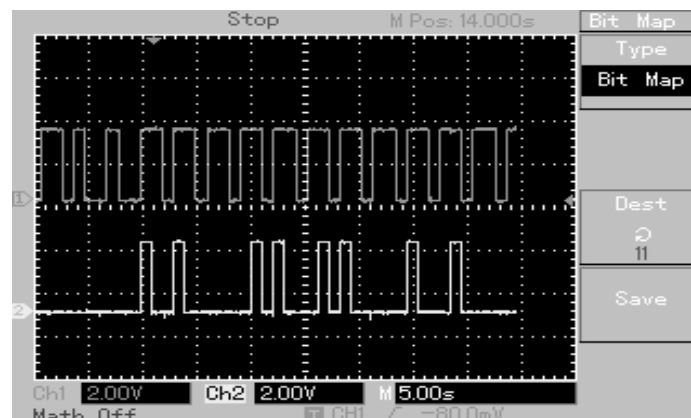
PROCEDURE:

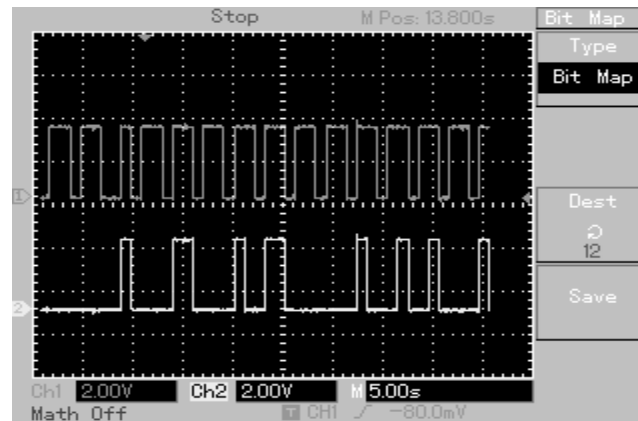
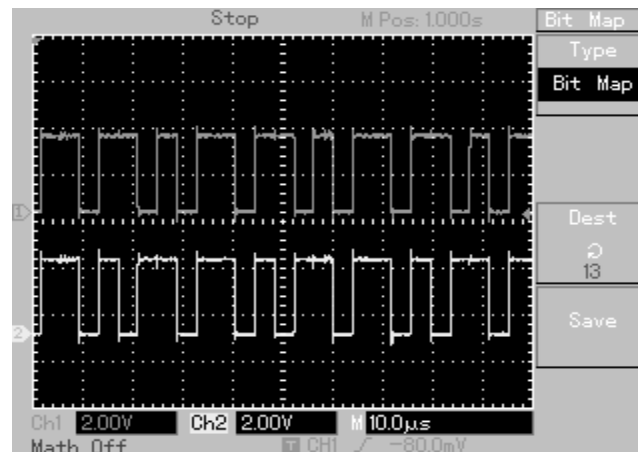
1. Refer to the Fig. 3.1 and carry out the following connections and **data clock switch** setting **SW2** in **slow** mode.
 2. Connect power supply in proper polarity to the kit **DCL-CONV** and switch it on.
 3. Select a data pattern using switch **SW1** of **data generator block** and connect SERIAL **DATA** generated on board to **DATA IN** of **convolution encoder**.
 4. Keep the jumper **J5** settings in the **puncturing** block as shown in Fig 2.1. Thus the Serial Data generated is connected to the input of convolution encoder and we can observe convolutionally encoded data at **IN3** and **IN4** post of **data puncturing** block.
 5. Observe the convolutionally encoded data at **IN3** and **IN4** of **DATA PUNCTURING** block.
 6. Observe the punctured convolutional code at **OUT3** with **DATA VALID1** and **OUT4** with **DATA VALID2** of **data puncturing** block. Note that a logic high on **DATA VALID1** and **DATA VALID2** indicates that there is a valid punctured data on **OUT3** and **OUT4** respectively.
 7. Connect **OUT3** and **OUT4** of **puncturing** block to **IN5** and **IN6** of **soft decision viterbi decoder** block.
 8. Observe the **depunctured data** in **soft signed magnitude** form at the **test points** provided in between **data depuncturing** and **viterbi decoder**.
 9. Observe the **decoded data** using soft decision viterbi decoder at **DATA OUT2** pin of the block.
- Note:** It is advised to observe the decoded data i.e. output of the soft decision Viterbi Decoder in fast mode because it takes approximately four minutes by the Viterbi decoder to decode the data since the operating frequency is very low.
10. Repeat the procedure by keeping the **data clock select** switch **SW2** to **fast** mode.
 11. Observe various waveforms as mentioned below.

OBSERVATION

Observe the following signal on the oscilloscope and you can verify it by solving, for the selected data pattern, using the steps given in the theory of this experiment.

- | | |
|--|----------------|
| 1. SERIAL DATA with respect to DATA CLK . | (FIG. 3.2 (a)) |
| 2. IN3 and IN4 . | (FIG. 3.2 (b)) |
| 3. OUT3 with respect to DATA VALID1 . | (FIG. 3.2 (c)) |
| 4. OUT4 with respect to DATA VALID2 . | (FIG. 3.2 (d)) |
| 5. DATA OUT2 with respect to SERIAL DATA . | (FIG. 3.2 (e)) |

CAPTURED WAVEFORM:**CH 1: DATA CLK (256 KHz OR 1Hz) & CH 2: SERIAL DATA (10101101)****FIG. 3.2 (a)****CH 1: IN3 & CH 2: IN4****FIG. 3.2 (b)****CH 1: DATA VALID1 & CH 2: OUT3****FIG. 3.2 (c)**

CH 1: DATA VALID2 & CH 2: OUT4**FIG. 3.2 (d)****CH 1: SERIAL DATA & CH 2: DATA OUT2****FIG. 3.2 (e)****CONCLUSION:**

Thus we have studied convolution encoding using puncturing and soft decision viterbi decoder using depuncturing technique and also observed how a serial data is convolutionally encoded and punctured using puncturing matrix and when it is transmitted using wire medium, and how it is decoded using soft decision viterbi decoding after depuncturing.

